AI Enhanced EHR System in Sri Lanka

- Final Year Thesis -

FACULTY OF COMPUTING AND
INFORMATION & TECHNOLOGY

SRI LANKA TECHNOLOGICAL
CAMPUS

FINAL YEAR PROJECT THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF
B.SC. HONS. IN SOFTWARE
ENGINEERING

July 2025

Dr.Azmeer Mohomed (Supervisor)

Ms.Rashmika Chandrasena (Co-Supervisor)

Dedication

I dedicate this thesis to my parents, who have always been my greatest source of strength. Their love, support, and guidance have helped me in every step of my education. They have made many sacrifices so that I could study and follow my goals. Their encouragement gave me confidence, especially during difficult times, and their belief in me kept me moving forward. I am truly thankful for everything they have done, and I would not have reached this stage without them.

To my project supervisor, Dr. Azmeer Mohommed and co-supervisor, Ms. Rashmika Chandrasena, your mentorship, expertise and steady guidance have shaped not only this project but also my growth as an aspiring engineer. Your influence will remain with me far beyond this work.

Declaration

I hereby declare that except where specific reference is made to the work of others, the contents of this dissertation are original and have not been submitted in whole or in part for consideration for any other degree or qualification in this, or any other university. This dissertation is my/our own work and contains nothing which is the outcome of work done in collaboration with others, except as specified in the text and Acknowledgments.

W.D.S.De Mel

W.A.C.Kavindya

H.A.L.Ruwanya

November 2025

Acknowledgements

I would like to express my sincere thanks to my supervisor, Dr. Azmeer Mohommed, for his guidance and support throughout this project. I also thank our co-supervisor, Ms. Rashmika Chandrasena, for her helpful feedback and advice.

My special thanks go to my team members, W.A.C. Kavindya and H.A.L. Ruwanya, for their hard work and cooperation during this project. I am also grateful to Sri Lanka Technological Campus for providing the resources needed for this study.

I would like to thank Panadura Hospital for sharing important information that supported our research. I also thank all healthcare professionals and participants who contributed their insights.

Finally, I am thankful to my family and friends for their continuous support and encouragement.

W.D.S. De Mel
SLTC Research Campus

Abstract

This study has presented the design and development of an AI-Enhanced Electronic Health Record (EHR) system aimed at addressing long-standing challenges in Sri Lankan healthcare, including fragmented patient records, handwritten prescriptions, limited interoperability and the absence of intelligent clinical decision-support tools. The system has been developed using a modular architecture consisting of a React-based frontend, a Node.js backend, a MongoDB database and a lightweight Python microservice. To ensure consistency and future interoperability, clinical data have been modelled using HL7 FHIR-aligned resource structures.

Two AI components have been incorporated into the system to enhance clinical workflows. The first is a clinical text summarisation module, which has generated concise summaries from encounter notes after applying strict de-identification to patient data. The second is a local rule-based prescription validation service that has identified allergy conflicts, duplicate medication entries and basic drug–condition mismatches. These AI modules have demonstrated the potential to improve decision-making and reduce common documentation and prescribing errors.

Functional and performance evaluations have shown that the system has supported complete clinical workflows, including patient registration, appointment scheduling, encounter documentation, digital prescription generation and QR-based prescription retrieval. Usability testing has indicated that users have found the interfaces clear and aligned with actual hospital processes. A stakeholder survey involving fifty-seven respondents has further highlighted the limitations of existing paper-based systems, the prevalence of problems with handwritten prescriptions and a strong preference for digital and AI-assisted healthcare solutions.

Overall, the study has demonstrated that an AI-enhanced, FHIR-aligned EHR system can be implemented effectively using open-source technologies and modest hardware. The results have shown that such a system can improve the clarity, accessibility and accuracy of clinical documentation while offering meaningful decision support in resource-constrained environments. The findings suggest that AI-driven digital health solutions have strong potential for adoption in Sri Lanka, and that the proposed system provides a practical foundation for future expansion and deployment at a national scale.

- .

Table of contents

1 Introduction 1

1.1 Background of the Study 1
1.2 Problem Statement 1
1.3 Aim of the Project 2
1.4 Objectives of the Project 2
1.5 Scope of the Study 3
1.6 Significance of the Project 3
1.7 Limitations of the Study 3
1.8 Summary of the Chapter 4

2 Related Work 5

2.1 Introduction to Related Work 5
2.2 Electronic Health Records (EHR) and Global Systems 5
2.3 FHIR and Interoperability 6
2.4 AI in Clinical Decision Support 7
2.5 Existing EMR Systems in Sri Lanka 7
2.6 Summary of Chapter 8

3 Methodology 9

3.1 Introduction to Chapter 9
3.2 System Architecture 9
3.2.1 High-Level Architecture 9
3.2.2 Backend Architecture 10
3.2.3 Frontend Architecture 10
3.2.4 Database and FHIR Modelling 11

3.2.5 AI Microservice Architecture 11
3.3 System Workflow and Process Design 12
3.4 Technology Selection and Justification 13
3.5 Summary of the Chapter 14

4 System Implementation and Experimental Setup 15

4.1 System Implementation Overview 15
4.1.1 Backend Implementation 15
4.1.2 Development Environment and Tools 16
4.1.3 Frontend Implementation 16
4.1.4 Database and FHIR Implementation 17
4.1.5 AI Microservice Implementation 17
4.1.6 Security and Authentication 18

4.2 Data Preparation and Test Scenario Design 19
4.2.1 Patient Data Preparation 19
4.2.2 Appointment and Encounter Data Preparation 19
4.2.3 Prescription Test Case Preparation 20
4.2.4 AI Test Case Preparation 20
4.2.5 Test Scenario Summary 20

4.3 Experimental Procedures and System Testing 21
4.3.1 Functional Testing 21
4.3.2 Backend and API Testing 21
4.3.3 Database and FHIR Resource Testing 22
4.3.4 Frontend Usability Testing 22
4.3.5 AI Summarisation Evaluation 23
4.3.6 Prescription Validation Evaluation 23
4.3.7 End-to-End Workflow Testing 24

4.4 Implementation Challenges and Workarounds 25
4.4.1 FHIR Modelling Challenges 25
4.4.2 Backend Integration Challenges 25
4.4.3 AI Microservice Challenges 26
4.4.4 UI/UX Challenges 26
4.4.5 Performance and Technical Limitations 27
4.4.6 Summary of Workarounds 27

5 Results and Analysis 28

5.1 Introduction to Chapter 28
5.2 Results 28
5.2.1 Functional Results 28
5.2.2 Backend and API Results 29
5.2.3 AI Feature Results 29
5.2.4 Usability Evaluation Results 30
5.2.5 Survey Results 30

5.3 Analysis 31
5.3.1 Achievement of Objectives 31
5.3.2 Significance of Findings 32
5.3.3 Comparison with Existing Work 32
5.3.4 Limitations Identified 33
5.3.5 Key Insights 33

6 Conclusions and Future Work 34

6.1 Conclusions 34
6.2 Future Work 35

References 36

LIST OF FIGURES

Figure 1.1: Overview of Healthcare Challenges in Sri Lanka
Figure 1.2: Proposed AI-Enhanced EHR System Architecture

Figure 3.1: Overall System Architecture
Figure 3.2: AI Summarisation Workflow
Figure 3.3: Local Prescription Validation Microservice Workflow
Figure 3.4: Complete System Workflow Diagram
Figure 3.5: Login Interface of the MediEase System
Figure 3.6: Doctor Dashboard Interface
Figure 3.7: Patient List Interface
Figure 3.8: Patient Overview and History Interface
Figure 3.9: Encounter Creation Interface
Figure 3.10: Visit Summary and AI Actions Panel
Figure 3.11: Prescription Creation Interface
Figure 3.12: Backend Folder Structure
Figure 3.13: FHIR Patient Resource Stored in Database
Figure 3.14: FHIR Prescription Resource Structure
Figure 3.15: Appointment Validation Workflow
Figure 3.16: User Registration and Approval Logic
Figure 3.17: Prescription–Encounter Linking Workflow
Figure 3.18: Python Microservice File Structure
Figure 3.19: Drug Classification Logic
Figure 3.20: Pydantic Schema for Validation API

Figure 4.1: System Deployment Architecture
Figure 4.2: Backend–AI Microservice Communication Flow
Figure 4.3: Example Output of AI Summarisation
Figure 4.4: Example Output of Prescription Validation Module

Figure 5.1: Age Distribution of Respondents
Figure 5.2: Exposure to Digital Record Systems
Figure 5.3: Problems with Current Paper-Based Systems
Figure 5.4: Problems with Handwritten Prescriptions
Figure 5.5: Preference for QR-Based Prescriptions
Figure 5.6: Acceptance of AI in Healthcare

A.1

Nomenclature

Acronyms / Abbreviations

BEM	Boundary Element Method
DEM	Discrete Element Method
FEM	Finite Element Method
FVM	Finite Volume Method
PFEM	Particle Finite Element Metho

Chapter 1

Introduction

The project history, scope, goals, and objectives are all explained in the introduction chapter. It explains the project's overall structure in detail.

1.1 Background

The Electronic Health Record (EHR) systems play a crucial role in the modern healthcare system since the system enables hospitals to store the information about the patients in a secure and systematic manner. Using a good EHR system, the doctors are able to view the medical history, treatments, lab reports, prescriptions, and notes of a person together. This will reduce the chances of information being lost, enable healthcare personnel to communicate with one another more effectively, and make more patient-safe decisions.

Most of the EHR systems in Sri Lanka perform only simple activities such as storing patient information, making visits, and posting reports in the digital records although many hospitals are beginning to use these records. Such rudimentary features are useful, yet physicians still need to read long records, find old data, and verify prescriptions on their own. All this requires a lot of time when done manually and may slow down the speed at which patients are served.

There is a need to expand the area of healthcare needs thus it is evident that EHR systems cannot merely store data. They must also assist the doctors to locate information very fast, maintain everything very well organized and simplify the details of the patients. The existence of better EHR systems can facilitate work of doctors and minimize errors, as well as assist hospitals in providing more credible care.

To assist in this, one of the goals of this project is to construct a more advanced EHR system with intelligent applications, such as automatic medical summaries and prescriptions checking. These new features will ensure that the doctors receive the appropriate and helpful information about the patients, and they will also assist the hospitals to work more effectively.

1.2 The Problem

Sri Lanka healthcare system has a number of issues that influence the pace, precision and safety of delivering care to patients. The following issues can be noted based on observations of existing practices in the hospitals:

Delayed Decision-Making in Emergency Cases

The physicians are unable to respond promptly in cases of emergency because of the inability to access all information about the patient. The records of the patients are disperse in various files or department and the doctors need to manually go through the files in order to know about the medical history. Such delay may influence the crucial decision-making and delay treatment exposing patients to danger.

Prescription Errors and Medication Mismatches

The use of manual methods of prescription raises the risk of making errors. Medical professionals have to use memory or consult through various records to prevent the issuance of incompatible drugs or wrong dosage. In the absence of automated support, the chances of medication errors are increased as they may affect patient safety.

Fragmented Patient Data Management

The information about the patient is distributed in various places and formats, and healthcare providers cannot easily have a complete picture of the medical history of a patient. Patient medical records are siloed in separate hospitals or clinics and they can not be used to coordinate the provision of holistic care to a patient as he or she visits various hospitals or clinics. This disintegration leads to unnecessary repeated tests, conflicting treatment and unutilized preventive services. Medical workers waste a lot of time in finding the records of patients, calling other hospitals, or using incomplete data given by patients themselves.

Limitations of Traditional Paper-Based Systems

In the majority of hospitals, paper records are still in use. Handwritten notes can be illegible, partial or spoilt. Documentation may be misplaced or lost and manual data typing may be erroneous. Physical file storage also causes a surge in administration work among doctors and nurses, which cuts down on the time that can be used to attend to the patients.

All these issues lead to a healthcare environment that is characterized by a risk of patient safety and delayed emergency care and decreased efficiency of the entire hospital. To solve these problems, a modern EHR system should be involved that can systematize patient data and introduce intelligent support to help doctors provide more rapid and safe care.

1.3 Proposed Solution

The proposed project will provide the solution to the problem of the existing healthcare system, offering to create an AI-Enhanced Electronic Health Record (EHR) System using an FHIR (Fast Healthcare Interoperability Resources) standard. The system should be created in a way that it is able to organize patient data effectively, offer smart assistance to physicians and enhance the quality and safety of healthcare provision.

It will automatically provide a summary of the medical history of a patient such as past diagnosis, treatments, lab results, and previous prescriptions. This enables the doctors to be able to know the status of the patient fast without having to go through long records in order to know the history of the patient thus saving time and effort in patient care.

Prescription Validation

The system will also contain the prescription validation module with AI. This characteristic will compare new prescriptions to the medical history of the patient to detect the risks that may cause it to be safer and more precise in handling medications.

FHIR-Based Data Management

With the FHIR standard, the system has guaranteed that patient data is recorded in an interoperable and structured format. This enables the doctors and healthcare facilities to gain easy access to accurate and complete patient information and the continuity of care is enhanced by eliminating fragmented data.

Advantages of the Proposed System.

• Quick decision making by doctors particularly during emergencies.

• Minor risk of prescription mistakes and drug misfit.

• Effective organization and summarization of medical history of the patient.

• Better patient safety and care in general.

• Less complicated transfer of patient data between departments and facilities.

The given AI-Enhanced EHR platform is a combination of the latest healthcare requirements and intelligent functions to meet the essential challenges of the hospitals of Sri Lanka and assist in the quick, safe, and efficient care of patients.

1.4 Deliverables and Milestones

The deployment of the AI Enhanced EHR system will be implemented in a coordinated and gradual process with clearly outlined deliverables and milestones. Such a programme schedule is a guarantee of systematic implementation of the project goals with the possibility of priori-iterative adjustment in accordance with testing and feedback.

Month 1: Initial Planning and Requirements Analysis

- Conduct stakeholder analysis including doctors, nurses, pharmacists and administrative staff
- Gather functional and non-functional requirements
- Analyze data requirements specific to Sri Lankan healthcare workflows
- Establish system scope, constraints, risks and success criteria
- Develop initial project documentation
- Create the Software Requirements Specification (SRS) document
- Deliverables:
    - SRS Document
    - Requirements Specification Report
    - Project Plan

Month 2: System Design

- Design full system architecture including client, server, database and AI layers
- Create detailed database schema for patient and clinical record management
- Develop complete UI/UX interface designs in Figma
- Create wireframes and high-fidelity prototypes for:
    - Patient information module
    - AI-assisted consultation notes
    - E-Prescription and pharmacy workflow
- Define technical specifications for each module
- Prepare API specifications and integration standards
- Deliverables:
    - System Design Document
    - Architectural Diagrams
    - Database Schema
    - Figma UI/UX prototypes

Month 3: Core Backend Development and FHIR Integration

- Implement full backend logic for hospital workflows
- Develop FHIR-compliant resources for:
    - Patient (PHN generation)
    - Practitioner
    - Appointment (APID generation)

    - Encounter (ENCID generation)
    - MedicationRequest
- Implement advanced lab management features:
    - Lab request creation (Doctor)
    - Lab report uploads (Lab Assistant)
    - Automatic generation of:
        - DiagnosticReport
        - Observation
- Integrate authentication and role-based access control:
    - Admin approval workflow
    - Doctor, Nurse, Lab Assistant permissions
- Implement REST API endpoints for all modules
- Perform structured backend testing using Postman/Newman
- Deliverables:
    - Fully functional backend with FHIR-compliant REST APIs
    - Working clinical workflows (patients → appointments → encounters → prescriptions → lab reports)
    - Updated technical documentation and Postman test suite

Month 4: System Integration, Clinical Workflow Completion and Testing

- During Month 4, the focus is on completing all major clinical workflows and ensuring that the full backend functions as a cohesive, stable and secure hospital EMR system.

Major Activities

- Integrate all system modules including:
    - Authentication and RBAC
    - Patient management
    - Appointment scheduling
    - Clinical encounters
    - Prescription management
    - Laboratory request and reporting system
- Ensure compliant implementation of all FHIR R4 resources:
    - Patient
    - Practitioner
    - Appointment
    - Encounter
    - MedicationRequest
    - DiagnosticReport
    - Observation
- Implement secure JWT authentication across all protected routes
- Ensure that the approval workflow prevents unauthorized access
- Begin full-scale testing using:
    - Postman automated collections
    - Integration test scenarios
    - Workflow validation across multiple user roles
- Verify behavior under typical Sri Lankan clinical scenarios (e.g., NIC-based identification)
- **Deliverables**
    - Fully integrated and functioning backend system
    - Working end-to-end clinical workflows

    - Postman test suite and test result reports
    - Updated API documentation

Month 5: Pilot Deployment, User Evaluation and System Refinement

- Month 5 focuses on deploying the system in a controlled testing environment, collecting feedback from real stakeholders and optimizing system performance and usability.

Major Activities

- Deploy system in a staged pilot environment
- Conduct user acceptance testing with:
    - Doctors
    - Nurses
    - Lab Assistants
    - Hospital Administration
- Evaluate:
    - Data flow
    - System usability
    - Clinical workflow efficiency
- Collect feedback through:
    - Observation
    - Structured interviews
    - Task-based performance measurement
- Identify system issues such as:
    - Validation gaps
    - UI workflow friction
    - Missing features
    - API-level inefficiencies
- Implement required improvements in:
    - API response speeds
    - Data validation accuracy
    - FHIR resource structure consistency
    - Error reporting and failure recovery
- Enhance performance monitoring and logging
- **Deliverables**
    - UAT report documenting system review findings
    - Updated system with refinements
    - Performance optimization report
    - Usability feedback summary

Month 6: Final Deployment, Documentation and Technical Consolidation

- Month 6 completes the deployment cycle and focuses on finalizing documentation and preparing the system for long-term use and evaluation.

Major Activities

- Deploy final stable system version
- Conduct structured training sessions for healthcare staff
    - Doctor onboarding

    - Nurse workflow training
    - Lab assistant usage
    - Admin management dashboard operations
- Ensure compliance with:
    - FHIR structural requirements
    - Sri Lankan medical data handling norms
    - Privacy and access control expectations
- Prepare complete system documentation, including:
    - Technical system documentation
    - Administration guides
    - User manuals
    - Maintenance and troubleshooting documentation
- Conduct post-deployment evaluation focusing on:
    - Performance reliability
    - System uptime
    - User experience feedback
    - Operational integrity
- Prepare final Thesis / Final Project Report
- **Deliverables**
    - Final deployed hospital management system
    - Complete software documentation package
    - Training materials and handover documents
    - Final project/thesis submission

Key Performance Indicators

- Reduction in time required to retrieve patient records
- Improvement in appointment-to-encounter processing speed
- Successful enforcement of access control and approval policies
- Higher accuracy in data entry due to field validation
- Reduced clinician workload through faster data access
- System uptime and reliability (e.g., ≥ 99% uptime during tests)
- User satisfaction across:
    - Doctors
    - Nurses
    - Lab staff
    - Admin personnel

This structured timeline with clearly defined milestones ensures accountability and allows for systematic progress tracking. Regular checkpoint reviews at the end of each month enable early identification of challenges and timely course corrections to keep the project on track.

Project Timeline (Gantt Chart)

Task	M11	M22	M33	M44	M55	M66
Requirements Analysis						
System Design						
Backend Development						
FHIR Integration						
Frontend Development						
AI Module Integration						
System Integration & Testing						
Pilot Deployment						
User Evaluation						
System Refinement						
Final Deployment						
Documentation						
Training						

Key Performance Indicators

Metric	Target
Patient record retrieval time	< 2 seconds
Prescription validation processing	< 3 seconds
Concurrent user support	Up to 100 users
System uptime during testing	≥ 99%
User satisfaction rating	≥ 4.0/5.0
Documentation time reduction	≥ 65%
Successful access control enforcement	100%
FHIR resource compliance	100%

Role-Specific Satisfaction Targets:

- Doctors: Improved prescription safety confidence
- Nurses: Reduced patient registration time
- Lab Assistants: Streamlined result upload workflow
- Administrators: Effective user management and audit capabilities

1.5 Outline of the report

This thesis is divided into six major chapters, the first of which is devoted to the most important aspects of the AI-Enhanced EHR system development and implementation:

Chapter 1: Introduction provides the context of the healthcare difficulties in Sri Lanka, defines the problems intended to be addressed by this study, describes the proposed AI-Enhanced EHR solution, and reports about the objectives of the project, deliverables and timeline.

Chapter 2: Related Work considers the current EHR systems, analyzing the methods of technological healthcare development in the world and in the developing countries. It examines the role of AI in clinical decision support, discusses the methods of prescription validation, assesses the existing level of EHR adoption in Sri Lanka, and outlines gaps that can prove the necessity of this study.

Chapter 3: Methodology details the systematic methodology of the development of the AI-Enhanced EHR system. It shows the general system structure and design, the stages of development, the AI algorithms and methods used, the issues of security and privacy, and explains why the technological decisions were made the way they were during the project.

Chapter 4: Experimental Setup and Implementation records the technical implementation of the system. It outlines the development environment and tools, outlines the database architecture and frontend interfaces and back end services, outlines how AI modules were integrated, outlines the challenges encountered during implementation and the solutions implemented, and outlines the testing procedures and framework.

Chapter 5: Results and Analysis gives the results of system development and evaluation. It evaluates the effectiveness of the AI summarization and prescription validation modules, measures system usability and user satisfaction, test scalability and reliability and compares the created system with the existing solutions.

Chapter 6: Conclusions and Future Work recaps the project accomplishments, outlines what this research contributed to the healthcare in Sri Lanka, what constraints the current implementation had, what additions and research directions can be made in the future, and what final thoughts can reflect on the outcome of the project.

Such a systematized structure helps the reader to direct a whole research process beginning with the identification of the problem to the design, implementation, and evaluation of the AI-Enhanced EHR system. The chapters develop one upon another creating a consistent story of the development of the system, its conception, and its validation within the Sri Lankan healthcare environment.

Chapter 2

Related work

Prior to coming up with any new system, it is paramount to know what has been achieved through the same research field. A related-work review gives such a background with the review of the current systems, methods and findings of previous researchers. Such a review enables the determination of the achieved achievements, the ongoing constraints, and the possible input of the existing research to the research sphere. Examining the information on Electronic Health Records (EHRs), interoperability standards, Artificial Intelligence (AI) applications, and prescription-safety mechanisms, one is able to understand the developmental process of digital healthcare systems and those aspects that need additional enhancement. In this chapter, the author has given an in-depth review of the existing research and outlined the gaps that informed the course of the current study.

2.1 Electronic Health Record (EHR) Systems

Electronic Health Records (EHR) Systems: EHR represents a computerized way to store and access health information (Carey et al., 2014).<|human|>2.1 Electronic Health Record (EHR) Systems

. Electronic Health Records (EHR): EHR is a computerized form of managing health records (Carey et al., 2014).

Electronic Health Record systems were developed to solve the issues that were prevalent in the paper-based records keeping. Paper files are easily lost, part or hard to locate easily when one is in need of important information. According to Thakkar and Davis, accessing records late, handwriting issues and fragmentation of information usually disrupt the quality of care in paper based settings. Digital EHRs would help to address these problems by offering quicker access to patient data, uniform documentation, and an enhanced level of coordination between health personnel..

The literature shows that the process of moving to the digital systems improves communication between departments and minimizes chances of manual mistakes. Nevertheless, there are still adoption problems. The initial cost of installation and maintenance cost cannot be afforded by some hospitals. Others do not have trained personnel to deal with digital systems. Most solutions that are currently used are storage systems, even with the implementation of EHRs, and do not incorporate any advanced features like real-time alerts or clinical decision support. Such restrictions can be viewed as an argument in favor of smarter and more efficient EHR solutions, especially in the countries that are just starting the digital transformation journey.

2.2 Risks, Barriers, and Challenges in EHR Adoption

Despite the many advantages of EHRs, their use is full of technical, financial, and even organizational difficulties. The high financial cost has been cited in the literature as the most common one. To sustain an EHR system, hospitals have to spend on servers, secure networks, software licenses, and trained IT labor force. Thakkar and Davis also observe that hospitals are not able to upgrade or increase their digital infrastructure due to budget constraints.

Adoption is also attributed to human factors. The healthcare professionals would be more fond of the paper-based system and reluctant to adopt the digital tools in their work in case they are afraid of the overload or committing some error. It requires proper training but most institutions have not been able to dedicate enough time to speak to training at all times.

Security and privacy factors make a significant contribution. According to Baumer et al., the sensitive patient information may be revealed due to the weak system design, or lack of access control, and hence, trust and even legal actions may be eroded. These results show that the implementation of EHR needs intensive policies and planning as well as safe implementation to achieve success in the long term.

2.3 Interoperability and the FHIR Standard

In modern healthcare, interoperability is critical since the information of patients may be shared between different institutions. Lack of adequate data sharing can lead to missing of entire medical histories by the clinicians and in turn delays and mistakes. Research proves that interoperability deficit leads to disjointed care, unnecessary tests, and lack of coordination.

To resolve this problem, Fast Healthcare Interoperability Resources (FHIR) standard was created. Ayaz et al. note that FHIR is a contemporary method of patient data exchange in healthcare based on the web-based tool of REST APIs, JSON, and XML. Resources are small and consistent medical information components within FHIR. Examples of such include a patient resource that includes demographic data, and an observation resource that includes vital signs or lab results.

FHIR has a number of benefits. First, it can be implemented by the developers easily since it adheres to common patterns of program used in web development. Second, it is versatile to the current uses as it can be used in mobile and cloud-based systems. Third, FHIR enhances standardization among hospitals and this helps to decrease inconsistency of medical information.

However, the studies also show challenges with the adoption of FHIR. Most health establishments do not have trained developers with the knowledge of the standard. This is because of the lack of coordination at the national-level in many countries, which has resulted in either partial or incompatible implementations. Irrespective of these difficulties, FHIR is the strongest candidate to bring the true interoperability to the healthcare sector and its benefits make it an appropriate tool in the given research.

2.4 Artificial Intelligence Applications in Healthcare

AI has become especially relevant in the field of healthcare since it helps to process a great amount of information and make decisions. AI is used to derive critical information in clinical text, patterns in patient history, and summarize long medical notes. Transformer-based models are better than older ones since they are able to comprehend complex and lengthy text better, as Taraghi et al. emphasize.

Another idea presented by the authors is modular AI architectures, which enable various components to be recycled across multiple tasks and simplify AI model training as well as adaptation to various data types. Even though their research does not point directly at the field of healthcare, the methods outlined are most applicable in the way of processing medical text, encounter notes, and diagnostic reports.

The paper also describes AI summarization as the one which saves the time clinicians devote to the reading of long documents. Nevertheless, the quality of the results will be high only with the help of domain-specific training data and fine-tuning. These are very much related to the present research where AI is employed to complement clinical operations by summarizing data that can quicken and make effective decision-making.

2.5 Prescription Validation and Medication Safety

Medication errors are also a huge threat to patient safety. There are possibilities of mistakes due to a combination of several medicines that negatively impact on the body, allergy, and incorrect doses. Prescription-safety studies show that manual checking is a slow process that is likely to have human error especially during the peak hour in the hospitals.

The authors outlay important elements of good prescription validation systems such as knowledge bases, interaction-checking algorithms and real time alert systems. These systems automatically match new prescription against patient history, medical instructions and recognized drug-interaction databases. In case a possible risk has been identified, the system notifies the clinician on time.

They show that automated prescription validation has a crucial effect on minimizing the risks of medication. They also point out that this type of systems works best when directly

embedded in an EHR system.

2.6 EHR Adoption in Sri Lanka

Sri Lanka is faced by many barriers to the integration of modern digital health systems. The use of paper records by hospital administrations is still a major hindrance to the promptness of patient care and the rapid accessibility of past medical information by clinicians. Delays like these can spark off disastrous consequences in cases of emergency situations of the dire nature.

Even in places that have digital infrastructures, they are often partial, isolated and not aligned to the current international standards including FHIR, causing disjointed information about the patient that is spread across different institutions. Further, there is a lack of implementation of AI-based systems to summarize, check prescriptions, or generate automatic reports. Clinicians have remained in the manual interpretation of huge documentation thus increasing workload and increasing the likelihood of human error.

The observations raise the point of a strong gap between the digital systems that are operational in Sri Lanka and the advanced technologies that are used in the world. Thus, the need to come up with smart, interoperable, and effective EHR solutions that could transform healthcare processes and support patient care is strong.

2.7 Gap in Existing Literature

The survey of the existing scholarship demonstrates a number of the missing links. Although the EHR systems are extensively studied, not many of them directly involve AI-based summarization or prescription validation into the clinical workflow. Much of empirical research on AI summarization is experimental with no real-life implementation of the technology in an EHR setting. On the same note, AI-based prescription validation has been studied in scholarly literature but has not been applied in the Sri Lankan clinical environment. None of the reviewed systems combine FHIR-compliant data structures and summarization with a localized AI safety engine into a single, unified EHR platform.

Such inadequacy confirms the fact that the given system creates some new contributions: The EHR should be integrated with AI summarization.

- Localized AI prescription validation microservice.
- A Sri Lankan healthcare-fitting system architecture based on FHIR.
- A hardware friendly AI implementation.

Such contributions directly tackle the specified gaps in the academic literature and address the issues that are especially relevant in developing countries.

2.8 Summary

The chapter analyzed the key literature related to Electronic Health Records systems, digital adoption obstacles, interoperability models, including FHIR, Artificial Intelligence apps, and prescription safety systems. The scholarship illustrates that although EHRs have significant improvement potential to healthcare delivery quality, cost, insufficient training, and poor interoperability have been limiting their influence. FHIR provides insightful basis of organized and conventional data exchange. AI provides a strong tool of clinical information processing, and prescription validation tools reduce the risk involved in medications. Sri Lanka, however, is not reliant on paper-based systems and is not using integrated and intelligent EHR systems. The results were used to establish the areas of research gaps that the current study aims to fill and give the necessary direction in developing a better healthcare system.

Chapter

Methodology

The research design is a methodology outlining how the systematic methodology was used to create and implement the AI-Enhanced Electronic Health Record (EHR) system. It describes the process by which an initial assessment of the healthcare problems in Sri Lanka was made to a functional solution of the integration of a typical EHR and AI-managed summarization and prescription validation. It required a strict approach to the methodology to make sure that the system addresses actual problems in terms of incomplete patient histories, handwritten records, work-intensive procedures, and a lack of decision-support systems at local hospitals.

This methodology has been developed to ensure a structured and logical unit, which begins with the identification of viable challenges in the current healthcare procedures, followed by the establishment of system requirements, and choice of suitable technologies. The chapter also clarifies the architectural planning, subsystem implementation, and the incorporation of AI parts into the workflow. The methodology focuses on developing a solution that is technically sound, user friendly and aligned with the limited hardware facilities that is the reality in most of the Sri Lankan healthcare environments.

This methodology provides the holistic view of the creation of the end EHR application by outlining the stages of design, development, and evaluation. It also shows how every decision is in line with the goals of improving patient information management, assisting clinicians in decision-making, and protecting the integrity of clinical data.

3.1 Research Design

The research design examines the general plan that will be followed in building the AI-Enhanced Electronic Health Record (EHR) system. It explains the way the system was developed in reaction to the real issues that were found in the Sri Lankan hospitals, i.e., the lack of patient history, hand-written notes, and the late access to the information in cases of emergencies. In line with this, the design is based on problem-based paradigm whereby every element of the system is carefully designed to address a certain clinical workflow requirement.

The design is more focused on adopting the FHIR standard of organizing all patient data thus ensuring that all encounters, diagnoses, and prescriptions are organized and systematically stored to enable system understanding and scalability. The AI capabilities are designed as central elements, not added elements; features like summarization and prescription validation will be their fundamental components since they will simplify the life of clinicians who have to manage massive amounts of written information and prevent risky choices concerning medication.

Practical limitations, including hardware limitations and high privacy standards are taken into account. As a result, summarization is done through an external HuggingFace API, and prescription validation is done on the local level through a Python microservice in order to maintain patient data confidentiality. Finally, the system is designed in a modular manner, where the frontend, backend, AI services, and database are divided, which increases the operational stability and makes the system easier to scale in the future.

3.2 System Architecture

System architecture defines how the major objects of the AI-Enhanced EHR system are configured and how these objects are connected. It has been designed to be simple, modular, and serviceable, but robust enough to serve the necessary AI power to perform summary and prescription validation. It uses a stratified design where every sub systems have a clear and unique purpose thus helping in understanding the system and enhancing reliability in the real clinical application.

There are four key components to the system. The first one is the frontend, which is used by the doctors and the staff. It allows individuals to search patients and make encounters, prescribe medications and read summaries or alerts. The second one is the Node.js backend that executes all the business logic. It retrieves the data on the frontend, verifies it, inserts it into the database and communicates with the services of the AI. The third component is the AI layer, which is further divided into two distinct components, the HuggingFace summarization service and a local Python microservice that verifies the prescription. The summarization service reduces the length of clinical notes to shorter text, and the local AI verifies risky medication and condition combinations. The last section is the Mongo database where all patient information is stored in the FHIR standard. The system is flexible as all elements communicate via API calls. Indicatively, the Python microservice can be requested by the backend to authenticate a prescription without disarranging other components. Figure 3.1 is a high-level diagram of this architecture, which demonstrates the flow of the information between the user and the backend to the AI services and lastly a database.

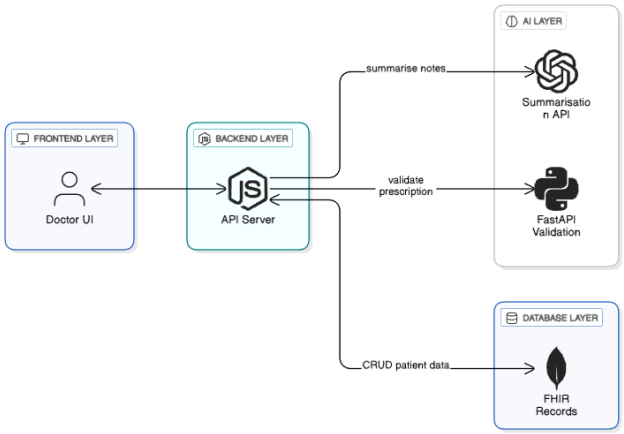


Figure 3.1 is a high-level diagram of this architecture

3.3 Methodological Workflow

3.3.1 Problem Identification and Requirement Gathering

This work began with the analysis of the issues within Sri Lankan healthcare. A lot of hospitals continue to rely on paper records and thus the doctors cannot access medical details promptly particularly during emergencies. Lost papers, absent documents, and the lack of any electronic safety measures increase the speed of the decision and increase the probability of errors. The analysis of these issues led to the identification of the key system requirements, which are digital patient records, structured encounters, AI summarization, and automated prescription validation. This step was the guiding point of the system as to what needed to be solved.

3.3.2Planning and Designing the System Architecture

The second step was to plan the system architecture after the requirements were clear. The design should be straightforward, modular and must be able to operate in low-resource environments. During this stage, the system was divided into frontend, backend, AI layer and database. The connections between these components were charted to retain data flowing well. The timing of this planning also allowed determining the manner in which each of the components would communicate with another and ensured that the system would be able to expand in the future without a complete redesign.

3.3.2 Backend Development Process

After the architecture was completed the backend development was started. The patient data were managed by the backend which was developed using Node.js and generated encounters, wrote prescriptions and generated records using the FHIR standards. The frontend was linked to the backend via API routes, and the logic was used to store and retrieve data in the database. It was also at this stage that the communication layer that connects both the backend and the AI summarization service as well as the local prescription validation microservice was added.

3.3.3 Frontend Development and UI Workflow

The frontend was next taken into consideration. The UI was created to resemble the way that physicians labor in a consultation session. Patient search, creation of encounters, prescription entry, and viewing AI results screens were created. Frontend communicates with the backend through API requests, allowing medical professionals to enter the data and receive in-the-moment summaries or warnings. The UI was made easy to use so that medical personnel do not have to be technical to make use of the system.

3.3.5 Integration of AI Summarization

The summarization functionality was introduced after the UI and the backend were stable. The backend retrieved encounter notes, deidentified them, and sent the clean text to the HuggingFace summarization service. The summary that was received was stored in the encounter record of the patient. This integration reduced the amount of time required to read through long histories by the doctors and assisted them in rapidly identifying important information about previous visits. Anonymization of data was essential in preserving patient confidentiality..

3.3.6 Implementation of Local AI Prescription Validation

This was followed by the local AI microservice to verify the prescription. It is written in Python using FastAPI and is used to classify medicines into drug classes and match them to known risk rules based on the conditions of the patient. The microservice receives and receives prescription data with the backend and receives warnings in case of identified risky combos. This step of the working process is effective to prevent medication errors and improve patient safety.

3.3.7 System Testing and Error Correction

After all components were completed, the entire system was then tested. Tests were used to ensure that the UI communicated with the backend correctly and that the data was stored correctly on the backend, as well as ensuring that both AI elements yielded correct results. Bugs or surprises that occurred were rectified. This process was continued until all the components of the system were functioning effectively and in a smooth manner.

3.3.8 Final Deployment and Evaluation

Final Deployment and Evaluation: The final deployment and evaluation are completed to verify that all issues related to the system are identified and resolved.<|human|>3.3.8 Final Deployment and Evaluation Final Deployment and Evaluation: Final deployment and evaluation are performed to ensure that all problems associated with the system are determined and addressed.

Last but not least, every part of the system was made to run with a real environment. Frontend, backend, AI services, and the database were launched locally and connected with the help of an entire workflow. The system was tested in terms of speed, accuracy and usability. This was to ensure that the entire solution was functional and fit to be used in the real world.

3.4 Node.js and React

The EHR application was developed using the Node.js back-end and a React-based front-end since these technologies are terrific to create a fast, reliable, and modern web application. The backend was developed using Node.js due to the fact that it provides a minimal and efficient environment to process API calls, process FHIR-based patient information, and communicate with AI services. As Node.js is built on a single-threaded and event-driven architecture, it can processing a large number of requests simultaneously without hosting it, which is ideal when it comes to health systems where nurses, doctors, and admins all interact with the system simultaneously.

The next factor that has made me choose Node.js is the massive library and tool base. It became much easier to organize API routes with the help of such modules as Express.js and restrict endpoints and integrate external services, such as the HuggingFace API and the Python microservice AI. The use of a JSON-native node.js also works well with MongoDB and FHIR which both operate on document-style data. This reduces the conversion time and will maintain the smooth flow of data between the data back-end and the database.

React has been selected as a front end since it assists in creating interactive responsive and component-based user interfaces. Within the hospital environment, the UI must be easy, quick, and user-friendly, and in particular, doctors and nurses are not willing to spend time in complex systems. React allows the UI to update immediately when data has been altered without having to load the entire page. This renders patient-to-patient switching a lot easier when it comes to adding encounters or even writing prescriptions.

The component-based architecture of React was also useful in maintaining a clean project. Redundant components such as common UI bits such as buttons, input fields, patient cards and tables were re-used. That reduced the time of development and maintained the appearance on each page. React is also compatible with REST APIs, and therefore, it was simple to integrate the frontend with the Node.js back end as well as the AI services. Consequently, the healthcare providers are able to receive AI briefs and prescriptive notifications promptly.

Node.js + React create a contemporary, high-performance development stack. Node.js supports rapid backend applications, glossy database interactions and simple AI incorporation. React provides a responsive and clean interface suitable to clinical workflows. These technologies enable us to construct a complete EHR system which is a pragmatic and expandable one in the future.

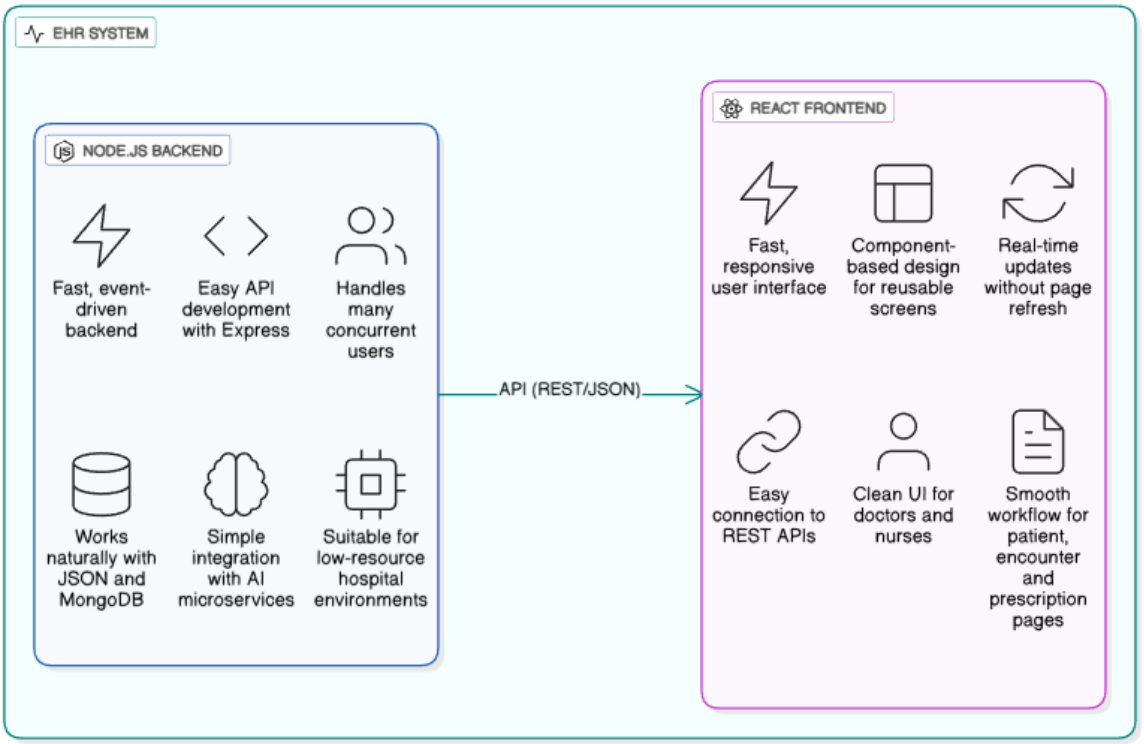


Figure 3.2: Justification for Using Node.js and React in the EHR System

3.5 Justification for HuggingFace Summarization

The choice of HuggingFace summarization API was based on its practical and technical considerations. Notes on medical encounters may be extremely long and difficult to read quickly, particularly when the doctor just requires a picture of the history of the patient in a minute. Summarization is beneficial because it makes the lengthy and incoherent text clear and concise. However, to operate a state-of-the-art summarization model on-site requires a high-end GPUs, of which most of the hospitals in Sri Lanka lack and were unavailable during this project. We therefore scoot over to HuggingFace API that provides access to powerful pretrained models that can vomit summaries without choking local machines.

The next reason why HuggingFace is big is the quality and reliability of its models. It would require a massive, specialized clinical dataset, and enormous amounts of time and computer power to train a medical summarizer. Listing HuggingFace provides already-trained stable transformer models on large text corpora, which allows us to create meaningful and accurate summaries without additional training. That renders the solution viable and achievable within the confines of a student project.

The first preference before utilizing an external API was one of privacy. To maintain patient data confidentiality, any text that is sent to us to be summarized has the personal identifiers such as names, PHN, address and age stripped out. Only the medical contents are

broadcasted. This will ensure there are no data leakages. The summarization remains safe and effective because of such intensive de-identification process.

Last but not the least is the feature of summarization which directly assists doctors. A concise summary of a patient in the past will enable a doctor to make faster and more successful decisions, particularly during emergencies. HuggingFace route is the most accurate and also the fastest and practical and thus it is the most suitable to use in this project.

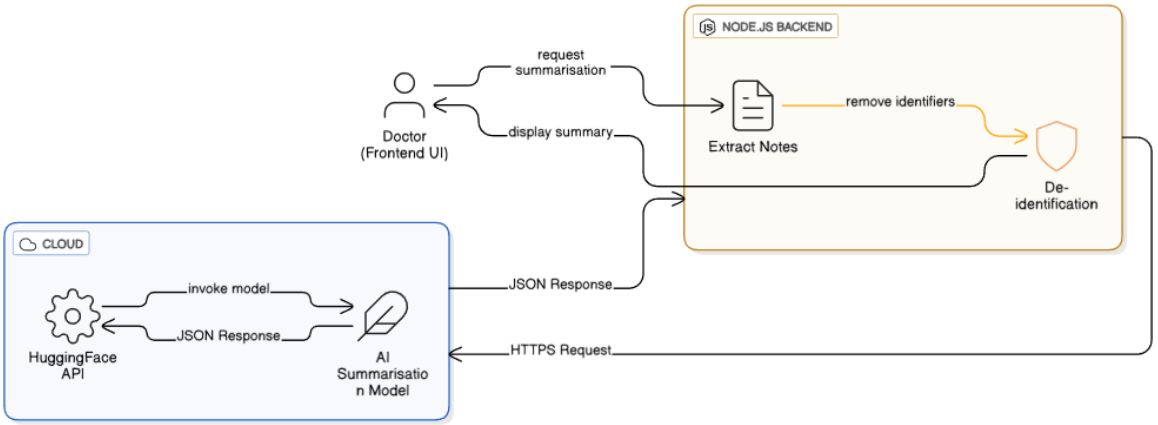


Figure 3.3: Placeholder for Cloud-Based Summarization Workflow Diagram

3.6 LLaMA-based medical summarization models

The summarization of long patient encounter notes is handled differently from prescription validation. For summarization, the system uses a model hosted on HuggingFace via their Inference API. This decision was made because running a high-quality summarization model locally requires significant computational power, which was not available for this project. Models such as **LLaMA-based medical summarization models or Meditron-7B** require:

- a dedicated GPU
- 16–24 GB VRAM
- fast storage
- strong CPU and RAM

The available hardware for this project was a normal mid-range computer (2–4 GB RAM for the AI service), which cannot run a 7-billion-parameter model.

Therefore, the summarization feature was implemented using the HuggingFace API.

3.7 Local AI Prescription Validation Method

This system prescription validation component was developed as an independent local microservice to enhance safety, decrease medical errors and support the doctor during prescribing. This microservice operates independently of the primary Node.js backend and is written in Python and FastAPI to scan against known-conditions of the patient to check the prescribed meds. The design ensures that the hospital is not dependent on cloud services to

perform vital safety functions hence the logic of validation can be executed even in case of failure of the internet.

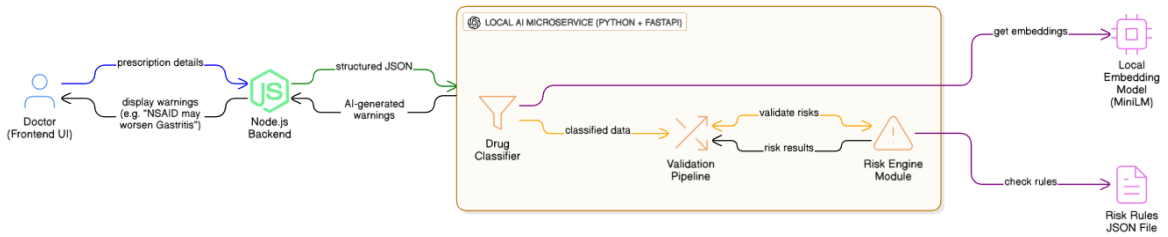


Figure 3.4: Placeholder for Local AI Prescription Validation Microservice Diagram

Internal components of the microservice are three in nature. The initial one is the drug classifier, which is based on a lightweight sentence-transformer model (MiniLM-L6-v2) that is used to transform names of medicines into embeddings and compares them with pre-trained examples of drug classes. The microservice can be installed on a standard computer (without a graphics card) because of its small size and friendlyness to the CPU.

The second module is the risk engine which is rule-based. A JSON file contains known clinical risks such as drug class and condition interaction such as gastritis, asthma or kidney disease. Upon a doctor making a prescription, the microservice joins the governed drug category to the ailments of the patient. In case a hazardous combination is discovered, a structured warning is released by the microservice. These warnings will contain the classes of drugs, the risky condition, the level of severity and a brief explanation.

The third module is the validation pipeline. It receives the request made by the main backend, invokes the classifier and the risk engine, synthesizes their responses and fits their responses and returns a clear response. The answer will be whether the prescription is safe or not and, in the event not, what dangers exist. Such an arrangement provides a readable, understandable AI-based decision support system that is not based on third-party cloud applications.

The microservice communicates using JSON in HTTP. A doctor, when developing a prescription on the frontend, packs the information and forwards it to the Python AI service on the back-end. The service calls out the meds and drops warnings that are immediately displayed in the UI of the doctor. The design maintains low latency and smooth workflow.

3.8 Overall System Workflow

This entire workflow demonstrates how everyone utilizes the AI-powered health system and how the information moves through all the steps of taking care of the patient.

When a patient enters the hospital a nurse or other personnel do the paperwork. The nurse will create a new patient account if he or she is new to using the program, or will make changes if the account exists. The nurse also schedules appointments for patients to meet with the doctor. The nurse's job is just registration and appointment stuff - no AI bits.

After the appointment has been made the doctor takes the reins. To read the patient's records and add notes while chatting, they have to log in first to prove that they're legit. While seeing the history, the doctor could ask the AI to provide a brief history of previous visits. The info is sent to the summarization API by the system and returns a short, clear recap which will help the doctor make a decision faster.

While seeing the patient, the doctor can give a prescription for meds. The prescription data gets sent to a little local AI helper that checks if the meds are safe for the patient's conditions. If there are any red flags, the system issues a warning to the doctor before the prescription is finalized. This helps to prevent medication errors and makes patient care safer.

The admin is like the manager. They approve new doctors, manage user accounts, and tweak system settings to everything runs smoothly without hiccups.

All together, patient, nurse, doctor, admin and the bits of AI work in a synchronised way. Everyone does their part and the AI steps in when you need it.

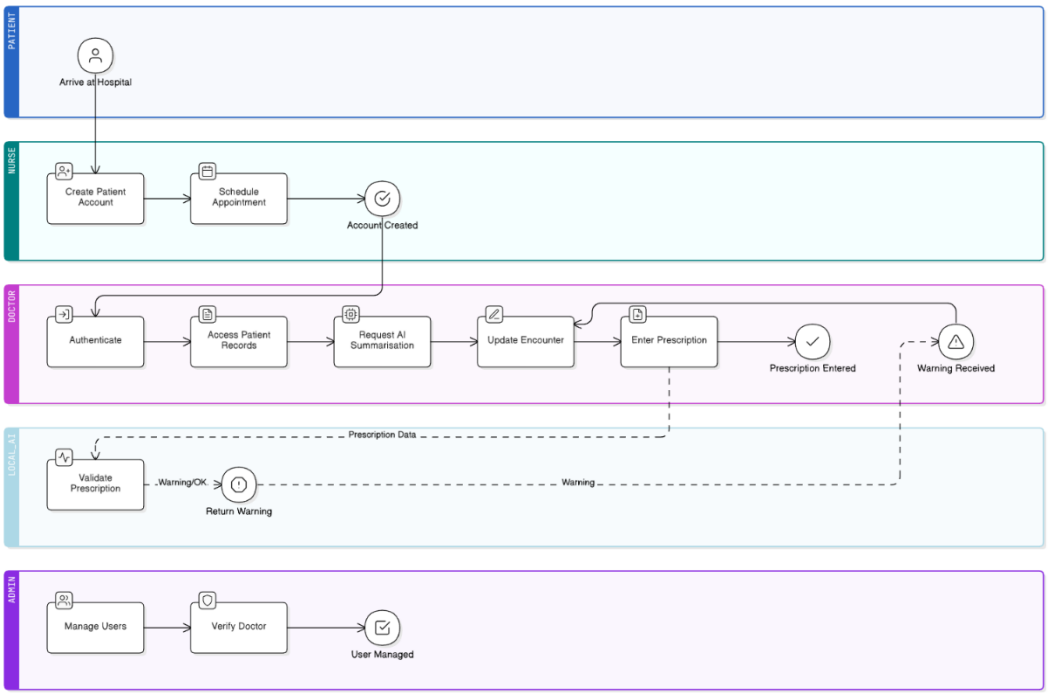


Figure 3.5: Overall System Workflow Diagram
This figure illustrates the complete workflow of the AI-Enhanced EHR system, including the interactions between the Patient, Nurse, Doctor, Admin and the AI services. The diagram visually represents patient registration, appointment handling, doctor authentication, record access, AI summarization requests, prescription validation and administrative management.

3.9 Security and Privacy Measure

Security and privacy was super important because the system deals with sensitive health info. The aim was to have patient data kept private and protected at all points.

First, there’s login. Only doctors, nurses and admins can get in. Each user has to log-in before he or she can see or change any records. That prevents the snooping of outsiders.

Second, any data sent to outside services will be scrubbed. The feature of summarization uses external API, therefore the system eliminates all personal details before sending text. That means that the outside service only sees the clinical part, but no names or addresses. The prescription checker remains local so no prescription or condition data ever leaves the system.

Third is the data storage is set up to maintain that privacy. All records are stored in MongoDB using the FHIR format - clear and organized. Only the backend communicates directly with the database; the front-end never communicates with it. All communication follows unambiguous API routes, so there's less potential for leaks.

These steps keep patient info safe and secret at all times.

3.10 Limitations and Assumptions

The project had a few real life limits that impacted what we could build.

The greatest limiting factor was hardware. I ran everything on a regular laptop with no dedicated GPU, and so couldn't train or run big transformer models locally. Stuff like the Meditron - 7B or Clinical LLaMA models need fancy GPUs with 16-24GB VRAM, fast storage and memory. Without that those models just didn't fit.

Because of that, we had to use HuggingFace API for summarization. The API allows us to make use of powerful models without local GPU power but requires an internet connection. In some parts of Sri Lanka where the internet is spotty, that could be a problem.

The prescription check is not without limits either. The local AI microservice is a small embedding model with a small rule set, which is used to identify drug-condition interaction. Making an entire medical knowledge with thousands of rules is not within the scope of a student project. So it can only catch the bleeding obvious risks. That's safer than nothing, but it can't replace a doctor's judgment with more complex interactions.

We also made some assumptions during development. The system presupposes that the input of nurses and doctors is accurate and complete. If a chronic condition isn't recorded, then the AI can't warn about related risks. We also assume that doctors will read AI warnings and use them as support, not as the sole decision makers.

These assumptions and limits provide the boundaries of what the system can do at this time. Another assumption is that medicines are entered using standard names so that the classifier can identify the correct drug class.

Finally, the system is still a prototype and was developed in a controlled environment. Actual hospital environments may involve more complex workflows, a larger number of users and heavier system loads. More extensive performance testing and optimisation would be required before the system can be deployed widely. Despite these limitations and

assumptions, the project successfully demonstrates how AI can support EHR functions and enhance clinical decision-making in a practical and resource-constrained environment

3.11 Summary

This chapter described the methodology used to design and develop the AI-Enhanced EHR system. It began by explaining the research approach and the reasoning behind the architectural decisions made during the project. The chapter then presented the full system architecture, covering the frontend, backend, AI components and database. Each part was designed to work together smoothly while remaining modular and easy to maintain.

The methodological workflow outlined the step-by-step development process, starting from identifying the problems in Sri Lankan healthcare, planning the system, building the user interfaces and backend services, integrating AI summarization, and implementing the local prescription validation microservice. The chapter also included detailed descriptions of the tools and technologies used, as well as the security and privacy measures applied to protect sensitive patient information.

Finally, the limitations and assumptions of the system were discussed, including hardware constraints, dependence on cloud summarization and the simplified rule-based validation engine. Despite these limitations, the methodology demonstrates that the system was developed logically and systematically, leading to a working prototype that supports clinical decision-making in a realistic and practical way.

Chapter 4 Experimental Setup and Implementation

This chapter explains how the AI-Enhanced Electronic Health Record (EHR) system was practically implemented. While Chapter 3 discussed the methodology and high-level design, this chapter focuses on how those ideas were realized in an actual working prototype. The aim is to describe the technical setup, tools, system structure and important implementation details without going into full code-level descriptions. Only the most critical parts of the system architecture, AI integration, backend structure and user interface development are discussed, as they directly contribute to solving the research problems identified earlier.

The chapter also presents the research instruments used, the data manipulation and testing process and the challenges faced during development. By the end of this chapter, the reader should clearly understand how the system was built, how the components work together and how the prototype was tested to ensure reliability.

4.1 Experimental Details

4.1.1 Prototype

The prototype of the AI-Enhanced Electronic Health Record (EHR) system was created and developed to a fully working app including frontend, backend, local AI microservice and database. Each component was developed independently and then connected through APIs so that the system could remain modular and expand. The prototype is the architecture from Chapter 3 that demonstrates how the design actually works in real life.

The frontend was created with the help of the React and provides a user interface to the doctors, nurses, and admins. It has a separate screen for login, patient registration, patient list, encounters, summaries and prescription creation. The UI was kept simple and quick to use so medical staff don't get stuck on techy stuff. Every time something happens in the frontend - say, making an encounter, generating a summary - it talks directly to the backend using secure http requests.

The backend is based on Node.js with Express.js to take care of the whole basic logic. It handles the login, FHIR-based patient records, storing the encounters, processing the prescriptions, and communicating with the A.I services. The backend is the main controller, sending encounter text to the summarization API and sending prescription details to the local AI microservice. All backend operations are based on the concept of

REST, thus, it's easy to test and add more functionalities.

The parts for AI are using a hybrid approach. The summarization feature uses the HuggingFace Inference API because the local hardware is unable to run a full medical summarization model. When a doctor requests a summary, the backend removes identifying information, sends the clinical text to the API, and stores the returned summary with the encounter. The prescription validation is the local Python FastAPI microservice. This service has a use case of drug classification using a light-weight embedding model and risk checking using a rule-based engine that is controlled by a set of rules contained in a set of JSON files.

MongoDB stores all patient data, encounters, summary, prescriptions and user details. The system uses FHIR resource formats for consistency, to prevent confusion and make future health care integrations easier. The DB stores documents in a flexible format in the form of a JSON document, which lends itself to the back end and artificial intelligence services.

The prototype was constructed as a piece of architecture step by step. The first version only had patient registration and basic encounter recording. After the core workflow was confirmed, AI summarization was added. Then, prescription validation microservice followed by better frontend screens came next. Each round made the prototype more functional, stable and user - friendly until the full workflow from patient registration to artificial intelligence (AI) supported decision making ran smoothly.

4.1.2 Frontend Implementation

The frontend of the MediEase Electronic Health Record (EHR) system was implemented using React.js and was designed to provide a clean, intuitive, and highly responsive interface for doctors, nurses, and administrators. Because clinical users interact with the system continuously during patient care, the frontend required a workflow-oriented design, minimal navigation complexity, and fast rendering. This section explains how the frontend was structured, how its components were organized, how routing and role-based access were implemented, and how the UI interacts with the backend and AI modules.

Component-Based Architecture

The frontend follows a fully modular and component-based architecture, which is one of the key strengths of the React framework. Each section of the user interface is broken into smaller, reusable components that can be combined to form complete pages. This approach reduces repetition, simplifies maintenance, and ensures consistent appearance throughout the application.

The codebase was divided into three main categories:

1. Components

These are reusable UI building blocks designed to be simple, independent, and flexible. Examples include:

- navigation sidebar

- top bar and header components
- patient summary card
- appointment list element
- encounter form field generators
- prescription input components
- modals for viewing or editing details

These components ensure visual consistency across different pages and allow future enhancements without major rewrites.

2. Pages

Pages are high-level screens that orchestrate multiple components to support specific workflows.

Examples include:

- **Login page:** for user authentication
- **Doctor Dashboard:** displays appointments and quick access to patient data
- **Patients page:** lists all patients and includes search/filter features
- **Patient Details page:** shows demographics, clinical history, and visit actions
- **Appointment page:** allows nurses to create or update appointments
- **Visit/Encounter page:** doctors record diagnoses, complaints, vitals, and treatment
- **Admin Panel:** used by administrators to approve/reject newly registered users

Each page loads relevant data from the backend and uses child components for layout and display.

3. Services (HTTP and Authentication Utilities)

A dedicated services folder contained utility modules responsible for communication with the backend API. A special HTTP client was created using Axios that automatically handled:

- attaching the JWT token to each request
- redirecting to login when the token expires
- setting a global base API URL
- unified error handling

This design ensures that individual pages or components are not responsible for handling authentication logic, making the codebase cleaner and more secure.

Routing and Navigation

Routing is an essential part of the frontend because different clinical roles need access to different workflows. React Router was used to implement routing, and routes were grouped based on user roles.

Doctor Routes:

- `/doctor/dashboard`
- `/doctor/patients`
- `/doctor/patient/:phn`
- `/doctor/encounter/:encid`
- `/doctor/prescriptions`

Doctors can view clinical information, start consultations, document encounters, and create prescriptions.

Nurse Routes:

- `/nurse/patients`
- `/nurse/patient-registration`
- `/nurse/appointments`

Nurses mainly perform registration and scheduling tasks.

Admin Routes:

- `/admin/users`
- `/admin/verify-doctor`
- `/admin/system-activity`

Administrators manage new accounts and system-wide configurations.

ProtectedRoute Implementation

A central authorization wrapper, often called **ProtectedRoute**, was implemented. This component checks:

- whether a valid JWT token exists
- whether the user's role matches the route’s allowed roles

If the checks fail, the user is smoothly redirected to the login page. This ensures that unauthorized users cannot access restricted areas even if they manually type the URL.

The ProtectedRoute pattern adds a robust security layer and aligns with HIPAA-style access control principles.

User Interfaces by Role

Doctor Interface

The doctor interface focuses on efficiency and clarity, supporting real clinical workflows such as:

- reviewing upcoming appointments
- viewing full patient medical history
- starting a patient encounter
- documenting chief complaints and diagnoses

- entering vital signs and clinical notes
- generating AI summaries (via HuggingFace)
- creating prescriptions and receiving AI risk warnings

This interface greatly reduces the time required for charting and decision-making.

Nurse Interface

The nurse interface provides streamlined workflows for:

- registering new patients
- editing patient demographic details
- scheduling appointments
- confirming appointments when patients arrive

Nurses do not have access to clinical diagnosis or prescription functions.

Administrator Interface

The administrator interface includes:

- reviewing newly registered users
- approving/rejecting doctor and nurse accounts
- managing system roles
- viewing system activity logs

This ensures that only verified medical staff access clinical features.

State Management and UI Behaviour

React hooks such as useState, useEffect, and occasionally useContext were used for state management. Global state was intentionally minimized to avoid complexity; only essential items like jwtToken and user role information were stored globally or in browser storage.

Front-end forms performed preliminary validation (required fields, correct formatting) before sending data to the backend. Tables and lists automatically refreshed after operations such as updating a patient or completing an encounter.

UI design principles focused on:

- reducing cognitive load
- displaying only relevant information
- maintaining consistency across all screens
- allowing fast clinician workflows

4.1.3 Frontend User Interface

The frontend of the MediEase EHR system was designed to provide medical staff with a clean, simple and easy-to-use interface that supports the daily clinical workflow. The UI was built to minimize cognitive load, reduce the number of clicks required to perform tasks and ensure that both doctors and nurses can complete their work efficiently. Each interface was carefully created to match the real-world sequence of activities in a hospital setting.

Login and Authentication Interface

The system begins with a role-based login page where users can sign in as a doctor, admin or staff member. This separation ensures access control and prevents unauthorised viewing of patient records. The login interface contains clear input fields, descriptive placeholders and visual selection cards for user roles.

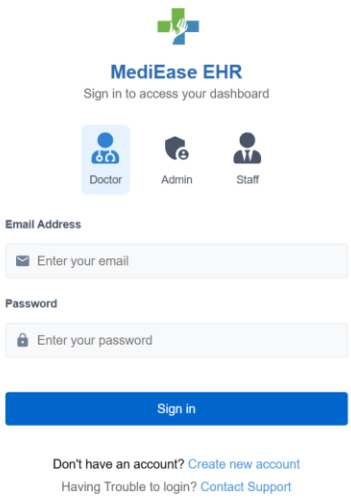


Figure 4.1: Login page of the MediEase EHR system

Doctor Dashboard Interface

After authentication, doctors are directed to a dashboard that shows upcoming appointments, recent patients and pending laboratory reports. This dashboard gives doctors an immediate overview of their daily workload and helps them prioritise urgent cases.

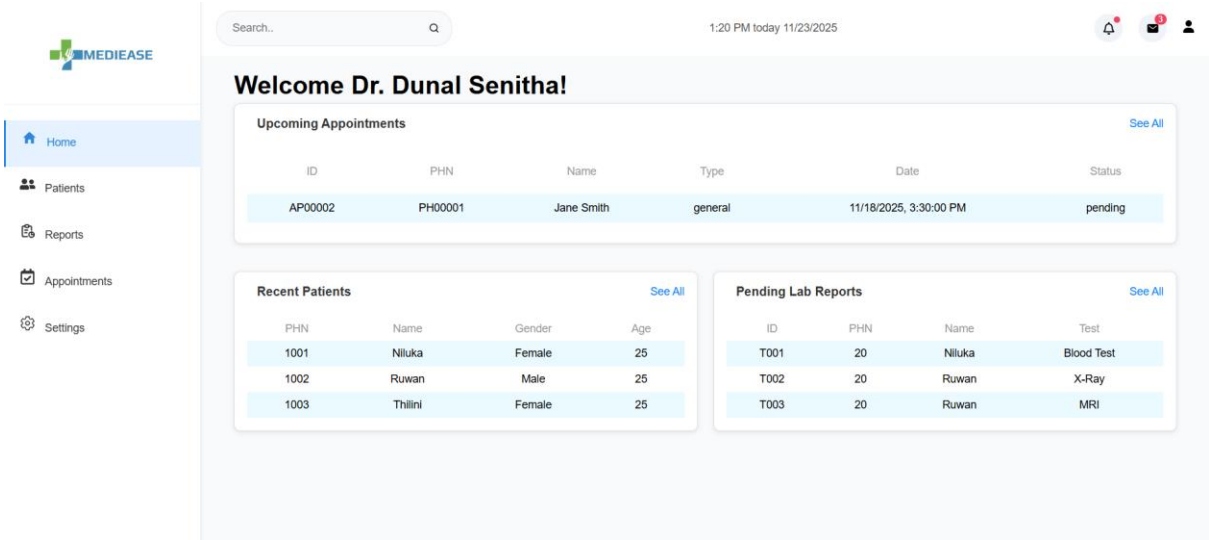


Figure 4.2: Doctor dashboard displaying appointments and patient overview.

Patient List Interface

The patient list screen displays all registered patients along with their PHN, name, NIC and nearest visit date. It allows staff or doctors to quickly search and select a patient for further actions such as viewing history or starting an encounter.

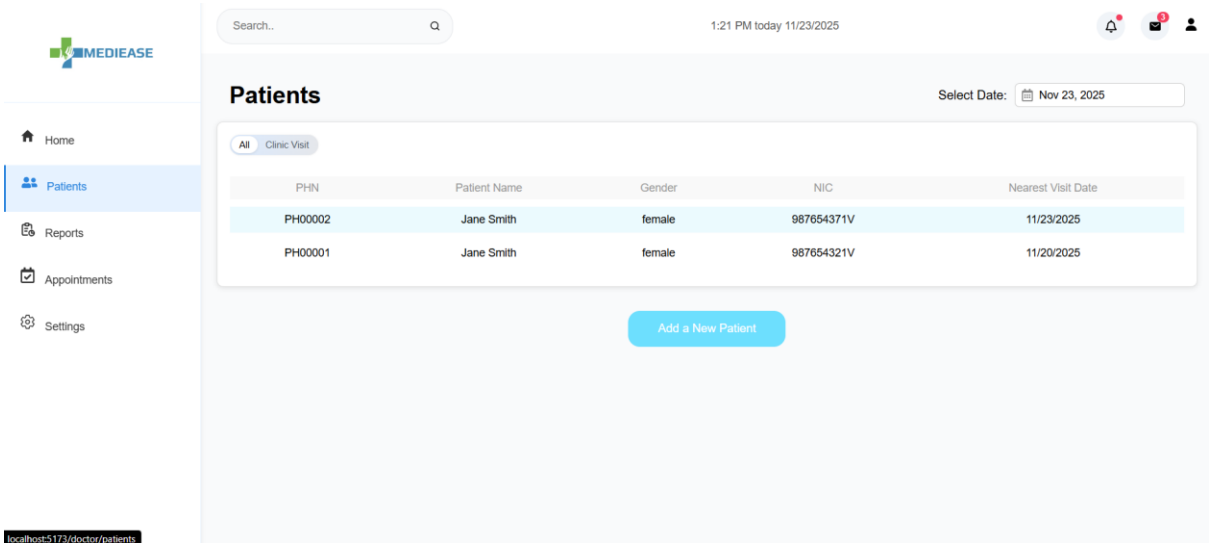


Figure 4.3: Patient list interface used for searching and selecting patients.

Patient Overview Interface

When a patient is selected, their detailed profile is shown. This page includes demographic data, address, contact details and general health information such as height, weight, BMI,

blood pressure and sugar levels. From here, the doctor can start a new clinic visit or update information.

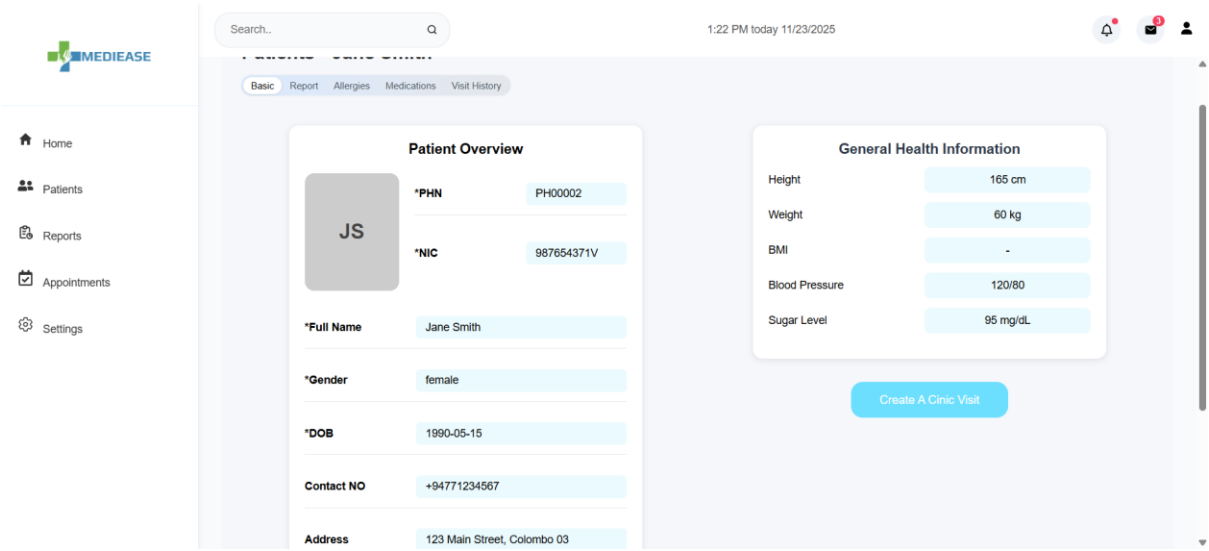


Figure 4.5: Patient overview showing demographic and general health information.

Encounter Creation Interface

The encounter page allows the doctor to start a new clinical visit by entering the patient's injury or complaint, weight, visit notes and other necessary details. This is where the primary clinical documentation begins.

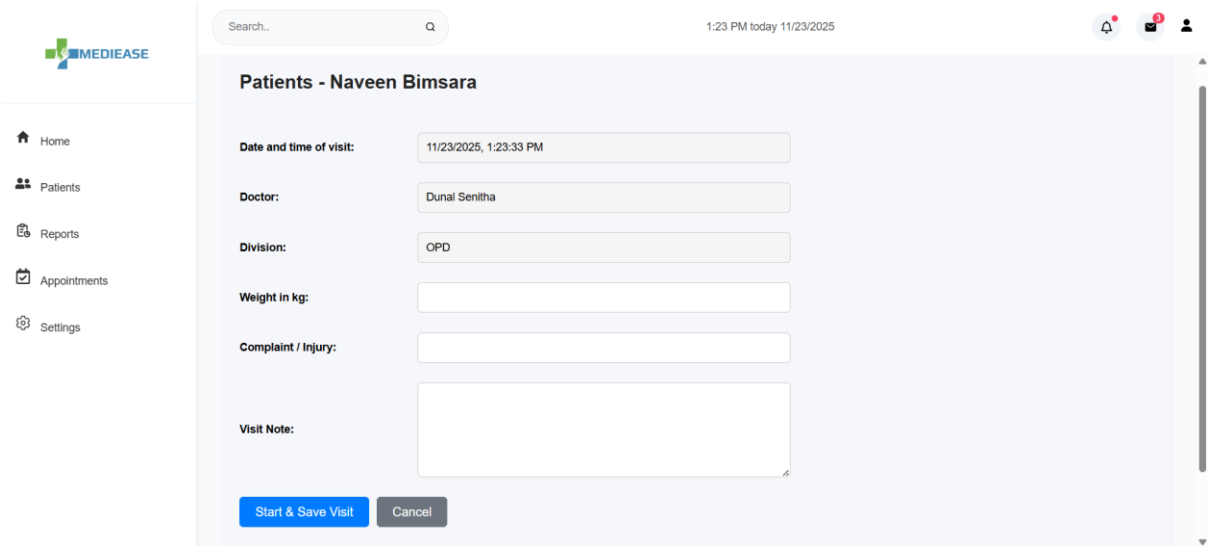


Figure 4.6: Encounter creation form for documenting the patient's condition.

Visit Summary and Actions Interface

After saving an encounter, the system displays a more detailed patient overview with additional actions such as ordering new lab reports, adding allergies, requesting AI summaries and starting a new prescription. This page captures the full clinical state of the patient at the time of visit.

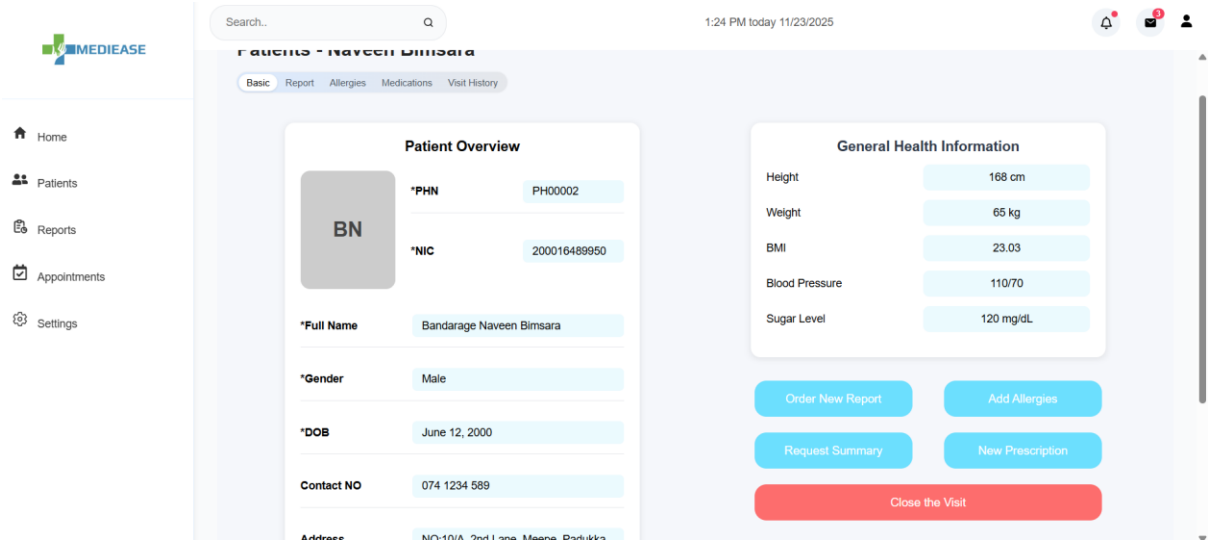


Figure 4.7: Visit summary interface with available clinical actions such as requesting AI summarization.

Prescription Creation Interface

The prescription page allows doctors to enter drugs by name, dose, frequency, period and custom comments. It includes options for favourite drugs and groups to make the prescribing process faster. This is also the interface where the AI prescription validation service returns warnings, helping doctors avoid mismatches with past conditions.

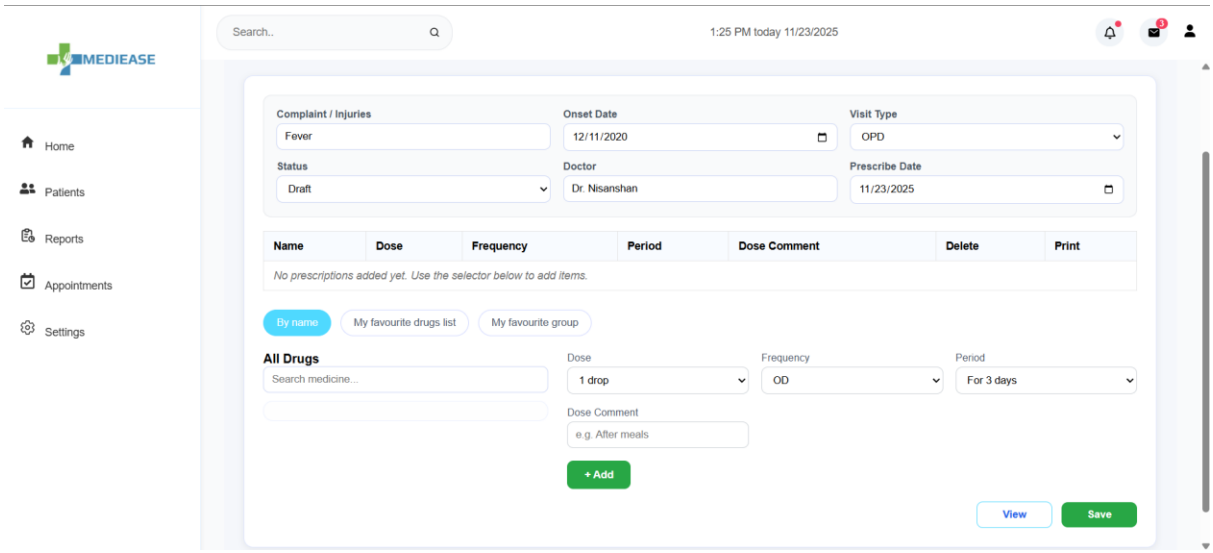


Figure 4.8: Prescription interface used by doctors to create and validate medication orders.

4.1.4 Database and FHIR Modelling

At the database layer of MediEase uses MongoDB with the help of Mongoose as the Object Document Mapper. Because EHR systems require speed and data consistency, the DB had to be designed using a hybrid model that stores every clinical object as a complete HL7 FHIR R4 resource, but pulls common query fields into top-level fields. This maintains interoperability with international standards without sacrificing query speed, which is essential for clinical tasks that need to be performed in real time.

Each main entity, i.e. Patient, Appointment, Encounter, Prescription is stored as a FHIR compliant JSON resource within a "resource" field. In addition to that complete resource, additional helper columns such as PHN, NIC, first name, last name, gender, appointment date, doctorId, and encounterId are contained in their own columns and indexed. For example, the Patient model captures both the full FHIR Patient resource and individual fields such as phn, nic, firstName and birthDate. That allows clinicians to rapidly search for patients based on common attributes and yet have all the detailed medical info available in the needed structured format. A similar pattern is used for appointments and encounters, with fields like apid, encid, patientPhn and status outside the main FHIR structure, so the system can get and filter big datasets fast.

Part of the big part of the DB design is to generate identifiers. All the core resources are using a sequential ID format - for example "PH00001" for patients, "AP00001" for appointments, "EN00001" for encounters. These ids are automatically generated using Mongoose pre save hooks. The hook verifies the highest existing ID in the collection and increments it, constructing it in the pattern. That way, every record gets a unique and readable ID that accommodates common hospital workflows. The generated ids are stored in the helper fields as well as inside the FHIR resource to ensure that internal operations and external integrations are in sync.

Another goal was to ensure everything remains fully auditable. Instead of deleting records, the system just updates the status of records. For example, appointments remain in the DB once a consultation is done; they're just flagged as 'completed' or 'cancelled.' Encounters and prescriptions retain their timestamps and references to related resources so that you can accurately reconstruct past clinical activity, which is important for a number of reasons, including patient safety and legal considerations.

Overall, the hybrid database and FHIR modelling strategy is a trade-off between performance and standardisation. MongoDB's flexible document structure allows the system to store big, nested FHIR resources without messing up, but the indexed helper fields allow the most common information to be pulled out quickly. As such, MediEase will be able to maintain FHIR-aligned data storage while still providing the speed and responsiveness required in daily clinical settings.

4.1.5 Backend Implementation (Node.js and FHIR)

The backend of MediEase was developed using node js and express js with modular architecture which divided model, controller, routes and middleware for neatness and reliability. Because the backend is the beating heart of the EHR system, it was important to

establish a stable data flow, stringent access controls, and complete HL7 FHIR standard alignment.

The type of models used in the backend of Mongoose, a hybrid approach of storing clinical information. Each record contains a full FHIR resource which is stored as a JSON string as well as simplified helper fields which make frequent queries faster and more efficient. For example, the Patient model has PHN, NIC, name and gender as indexed fields but the complete Patient FHIR structure is maintained for interoperability. This way, the system could leverage the FHIR compatibility without compromising the performance needed for real-time clinical workflows.

Controllers are the major logic in the backend. The authentication controller is responsible for user registration and login and administrative account approval. Passwords are hashed in a secure way and on successful logins, a JSON Web Token is generated which is used by the frontend for all protected API calls. Creation and retrieval of patient records, and automatic assignment of sequential PHN numbers are performed by the patient controller by using a pre-save hook. Appointment management is performed using a separate controller specifically for creation and modification of appointments, where appointments are entities linked to two entities, patients and practitioners. The encounter controller manages the clinical documentation process by creating a new encounter when a doctor starts a consultation and updating the encounter with clinical findings such as vital signs, observations and diagnoses.

Prescriptions are processed in a similar manner via a dedicated controller which builds a MedicationRequest resource and associates it with the encounter.
Routes define the entry points of the backend and are structured to separate different areas of the system such as authentication, patients, appointments, encounters and prescriptions. Middleware ensures that only authenticated users can access the protected routes and each route ensures that appropriate role-based permissions are enforced. For example, doctors can create encounters and prescriptions, nurse can register patients and schedule appointments, and administrator can approve new users.

Error-handling middleware is used to ensure that unexpected errors occurring in the backend are returned in a controlled and consistent format.
FHIR resource construction is managed through the helper functions that were designed to create standard-compliant objects. This way, repetition is avoided and inconsistencies in structures of resources in different controllers also prevented. The integration of AI is supported in the backend as well. Clinical summaries are generated with the HuggingFace API, but before external note sends, the backend strips away identifying information in order to keep the notes private. Prescription validation is taken care of by the local AI microservice; the backend prepares the medication and condition data, sends it for evaluation and returns alerts to the doctor if any.

The backend provides a list of structured, Restful API endpoints that provide functionality for authentication, patient management, appointment workflows, encounter creation and prescription handling. These endpoints adhere to the same naming convention and they return the responses in a format that is compliant with FHIR R4 structures and is in a format that is easy to parse as a result.

Authentication Endpoints

- POST /api/auth/register – Registers a new doctor or nurse account.
- POST /api/auth/login – Authenticates a user and returns a signed JWT token.
- GET /api/auth/me – Retrieves the currently authenticated user's profile.
- PUT /api/auth/approve/:userId – Allows administrators to approve pending users.

Patient Endpoints

- POST /api/patients – Creates a new patient record and generates a PHN.
- GET /api/patients – Retrieves all patients with search and filtering options.
- GET /api/patients/:phn – Retrieves a specific patient by PHN.
- PUT /api/patients/:phn – Updates demographic information for an existing patient.

Appointment Endpoints

- POST /api/appointments – Creates a new appointment (nurse role).
- GET /api/appointments?date= – Retrieves appointments filtered by date.
- GET /api/appointments/doctor/:doctorId – Lists all appointments assigned to a doctor.
- PUT /api/appointments/:id/status – Updates appointment status (e.g., completed, cancelled).

Encounter Endpoints

- POST /api/encounters – Creates a new clinical encounter linked to an appointment.
- GET /api/encounters/patient/:phn – Retrieves encounter history for a patient.
- GET /api/encounters/:encid – Retrieves a specific encounter by ID.

Prescription Endpoints

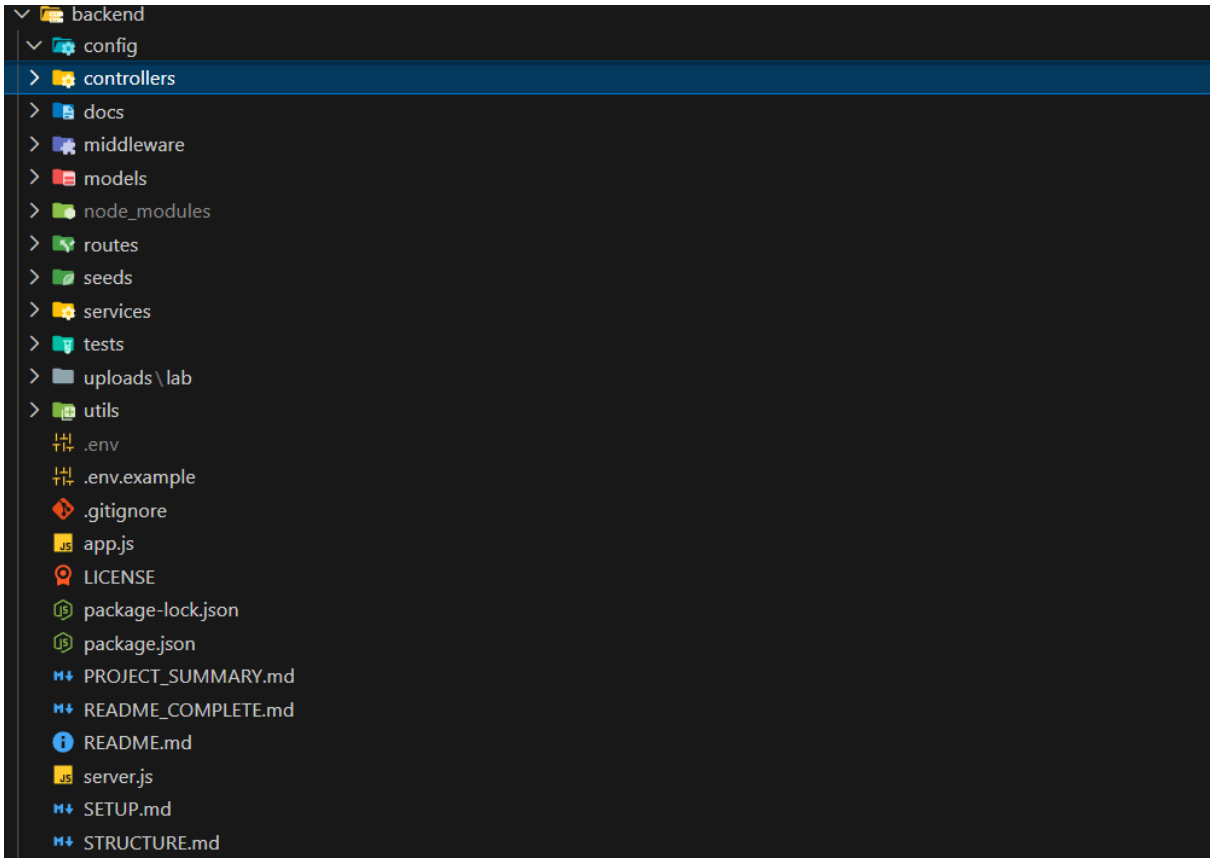
- POST /api/prescriptions – Creates a prescription and links it to an encounter.
- GET /api/prescriptions/encounter/:encid – Retrieves all prescriptions for a specific encounter.
- GET /api/prescriptions/patient/:phn – Retrieves all prescriptions associated with a patient.

AI Integration Endpoints

- POST /api/ai/summarise – Sends encounter notes to the summarization module after de-identification.
- POST /api/ai/validate – Sends prescription details to the local AI microservice for drug–condition risk analysis.

The overall backend structure, including the FHIR-specific controllers and models, is shown in Figure 4.X.

Figure 4.9: Backend folder structure with config, controllers, middleware, models and routes



Each core FHIR resource used in the system has its own controller file, such as `fhirPatient.controller.js`, `fhirEncounter.controller.js`, `fhirAppointment.controller.js` and `fhirPrescription.controller.js`. These controllers contain the logic for creating, reading and updating FHIR documents in MongoDB. For example, the patient controller creates a new patient by combining basic metadata (such as PHN, name, gender and contact details) with a full FHIR Patient resource. After validation, the resource is saved and a structured JSON response is returned to the client. This pattern ensures that the system always stores FHIR-compliant data while still providing simple responses for the frontend to consume.

Figure 4.10: Example controller logic for creating a patient with a FHIR resource and metadata

```
// Create patient with FHIR resource
const patient = await FHIRPatient.create({
  ...patientData,
  phn,
  resource: fhirResource
});

res.status(201).json({
  success: true,
  message: 'Patient created successfully',
  data: {
    id: patient._id,
    phn: patient.phn,
    resource: patient.resource,
    metadata: {
      firstName: patient.firstName,
      lastName: patient.lastName,
      gender: patient.gender,
      birthDate: patient.birthDate,
      contactNumber: patient.contactNumber
    }
  }
});
});
```

The encounter and appointment flow is also carefully controlled by the backend. When a doctor tries to start an encounter, the backend first checks the related appointment. It verifies that the appointment exists, has not already been completed and has not been cancelled. It then confirms that the authenticated doctor matches the doctor assigned to that appointment. If any check fails, a clear error message is returned. This prevents encounters being started under the wrong appointment and enforces correct workflow usage.

Figure 4.11: Appointment validation logic ensuring only valid and active appointments can start encounters

```
// Find appointment
const appointment = await FHIRAppointment.findOne({ apid });
if (!appointment) {
  return res.status(404).json({
    success: false,
    message: `Appointment not found with APID: ${apid}`
  });
}

// Verify appointment status
if (appointment.status === 'completed') {
  return res.status(400).json({
    success: false,
    message: 'This appointment has already been completed'
  });
}

if (appointment.status === 'cancelled') {
  return res.status(400).json({
    success: false,
    message: 'Cannot start encounter for cancelled appointment'
  });
}

// Get doctor's medical license from authenticated user
const doctorLicense = req.user.medicalLicenseId;

// Verify doctor matches appointment
if (appointment.doctorLicense !== doctorLicense) {
  return res.status(403).json({
    success: false,
    message: 'You can only start encounters for your own appointments'
  });
}
```

User registration and role management were handled in the authentication controller. During registration, the backend validates that all required fields are present, checks that the role is one of doctor, nurse or lab assistant and enforces additional constraints such as requiring a medical licence ID when the role is doctor. Password confirmation is also checked before saving the user. These checks ensure that only correctly defined users can enter the system and that each user's role is properly configured.

Figure 4.12: User registration controller with validation for roles, passwords and medical licence ID

```
export const register = asyncHandler(async (req, res) => {
  const { firstName, lastName, email, password, confirmPassword, role, medicalLicenseId, nurId, labId, nic, division } = req.body;

  // Validate required fields
  if (!firstName || !lastName || !email || !password || !confirmPassword || !role || !nic) {
    return res.status(400).json({
      success: false,
      message: 'Please provide all required fields'
    });
  }

  // Validate role (now includes lab_assistant)
  if (!['doctor', 'nurse', 'lab_assistant'].includes(role)) {
    return res.status(400).json({
      success: false,
      message: 'Role must be doctor, nurse, or lab_assistant'
    });
  }

  // Validate password match
  if (password !== confirmPassword) {
    return res.status(400).json({
      success: false,
      message: 'Passwords do not match'
    });
  }

  // Check if doctor and validate medical license
  if (role === 'doctor' && !medicalLicenseId) {
    return res.status(400).json({
      success: false,
      message: 'Medical License ID is required for doctors'
    });
  }
```

To guarantee that incoming FHIR data is structurally valid, a dedicated middleware component was implemented. This middleware extracts the `resource` field from the request body, verifies that it exists and then passes it through a validation function. If the resource does not meet the required structure, the middleware returns a 400-level error with specific validation messages. Only valid resources proceed to the corresponding controller. This approach centralises FHIR validation and prevents invalid documents from entering the database.

Figure 4.13: Middleware for validating the basic FHIR resource structure in incoming requests

```
* Middleware to validate basic FHIR resource structure
*/
export const validateFHIRStructure = (req, res, next) => {
  const { resource } = req.body;

  if (!resource) {
    return res.status(400).json({
      success: false,
      message: 'FHIR resource is required in request body'
    });
  }

  const validation = validateFHIRResource(resource);

  if (!validation.valid) {
    return res.status(400).json({
      success: false,
      message: 'Invalid FHIR resource structure',
      errors: validation.errors
    });
  }

  next();
};
```

Another important part of the backend implementation was the link between encounters and prescriptions. Before a new prescription is created, the backend checks that an active encounter exists for the given patient and doctor. If no such encounter is found, the backend returns a message instructing the doctor to create an encounter first. Once a prescription document is created successfully, a helper function attaches the prescription's ObjectId to the existing encounter and marks that encounter as having a prescription. This ensures that every prescription is tied to a specific clinical visit and can be traced back easily.

Figure 4.14: Helper functions validating encounters before prescriptions and attaching prescriptions to encounters

```
10 export async function validateEncounterBeforePrescription(patientPhn, medicalLicenseId) {
11   const encounter = await FHIREncounter.findOne({
12     patientPhn,
13     doctorLicense: medicalLicenseId,
14     isActive: true,
15     status: { $nin: ['finished', 'cancelled'] }
16   });
17   if (!encounter) {
18     return {
19       valid: false,
20       message: 'Doctor must create an encounter before adding a prescription.'
21     };
22   }
23   return { valid: true, encounter };
24 }
25
26 /**
27  * Mark encounter as having a prescription and attach prescription ObjectId.
28  * @param {string} encounterId MongoID of encounter document
29  * @param {string} prescriptionId MongoID of prescription document
30  */
31 export async function attachPrescriptionToEncounter(encounterId, prescriptionId) {
32   await FHIREncounter.findByIdAndUpdate(
33     encounterId,
34     {
35       $set: { hasPrescription: true },
36       $addToSet: { prescriptions: prescriptionId }
37     },
38     { new: true }
39   );
40 }
```

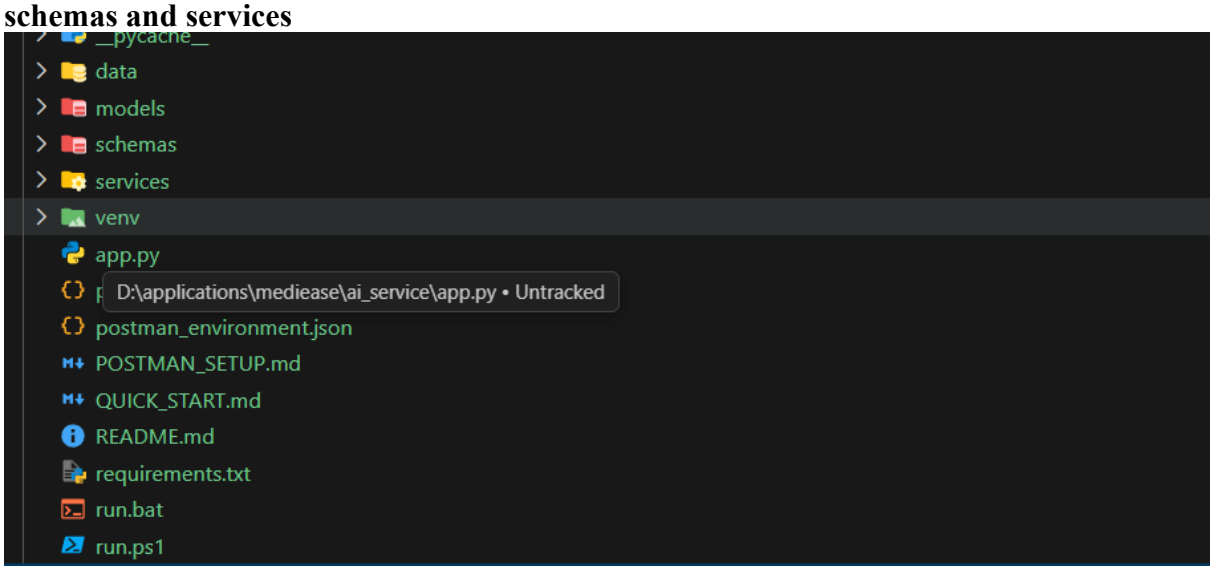
Overall, the backend implementation realises the planned architecture by combining FHIR-compliant models, strict validation, clear separation of concerns and a well-defined workflow for appointments, encounters and prescriptions.

4.1.4 AI Prescription Validation Microservice (Python FastAPI)

The prescription validation logic was implemented as a separate Python microservice using FastAPI. This microservice runs locally and communicates with the Node.js backend over HTTP. Its main purpose is to classify medicines into drug classes using sentence embeddings and to check those classes against a rule-based risk engine.

The microservice is organised into folders for data, models, schemas and services, as well as a main `app.py` entry file. The folder structure is shown in Figure 4.X. The `data` directory contains JSON files with drug class examples and clinical risk rules. The `models` folder contains the classifier implementation, while `schemas` defines the Pydantic request and response models. The `services` folder implements the validation pipeline that orchestrates classification and risk detection.

Figure 4.15: Folder structure of the Python AI microservice, including data, models, schemas and services



The classifier loads a small sentence-transformers model (MiniLM) at startup. It also reads the class example JSON file and precomputes embeddings for each example. When the service receives a medicine name, it encodes the text and compares it to the stored class embeddings. The class with the highest similarity above a threshold is selected as the predicted drug class. The loading of the model and class examples, together with logging and error handling, is shown in Figure 4.X.

Figure 4.X: Classifier code that loads the sentence-transformers model and precomputes embeddings for drug classes

```python
def load_model(self):
    """Load the sentence-transformers model."""
    try:
        logger.info("Loading sentence-transformers model...")
        self.model = SentenceTransformer(self.model_name)
        logger.info("Model loaded successfully")
    except Exception as e:
        logger.error(f"Failed to load model: {e}")
        raise

def load_class_examples(self, examples_path: str):
    """
    Load drug class examples from JSON file.

    Args:
        examples_path: Path to the class_examples.json file
    """
    try:
        logger.info(f"Loading class examples from {examples_path}")
        with open(examples_path, 'r', encoding='utf-8') as f:
            self.class_examples = json.load(f)

        logger.info(f"Loaded {len(self.class_examples)} drug classes")

        # Precompute embeddings for all class examples
        self._precompute_class_embeddings()

    except FileNotFoundError:
        logger.error(f"Class examples file not found: {examples_path}")
        raise
    except json.JSONDecodeError as e:
        logger.error(f"Invalid JSON in class examples file: {e}")
        raise
```

The request structure for validation is defined using Pydantic schemas. The `ValidatePrescriptionRequest` model includes the patient PHN, list of conditions, current complaint and an array of medicines. Each medicine is described using the `MedicineItem` model, which contains fields such as name, optional RxNorm ID, dose, frequency, period and instructions. This schema ensures that the incoming data from the Node.js backend is well structured and that all required information is present. An example request is embedded inside the schema definition to make the API easier to understand and test.

Figure 4.X: Pydantic request schema for prescription validation, including example JSON payload

```python
def get_classifier() -> DrugClassifier:
    """Get the global classifier instance."""
    if classifier_instance is None:
        raise RuntimeError("Classifier not initialized. Call initialize_classifier() first.")
    return classifier_instance


def initialize_classifier(model_name: str = "sentence-transformers/all-MiniLM-L6-v2",
                          examples_path: str = "data/class_examples.json"):
    """
    Initialize the global classifier instance.

    Args:
        model_name: Name of the sentence-transformers model
        examples_path: Path to class examples JSON file
    """
    global classifier_instance

    classifier_instance = DrugClassifier(model_name)
    classifier_instance.load_model()
    classifier_instance.load_class_examples(examples_path)

    logger.info("Classifier initialized successfully")
    return classifier_instance
```

When the Node.js backend sends a prescription to the microservice, the validation service first uses the classifier to determine the drug class for each medicine. It then checks those classes against the risk rules associated with each patient condition. If a risky combination is found, the service generates a warning object containing the medicine name, drug class, related condition and a message describing the risk. These warnings are returned as a JSON response to the backend, which forwards them to the doctor. Because the microservice runs locally, no prescription or condition data leaves the system.

Through this implementation, the prototype demonstrates that a lightweight local AI service can provide meaningful safety checks without requiring high-end hardware or external cloud dependencies

4.1.6 Development Environment and Tools

The development of the MediEase system was carried out using a carefully selected set of tools, frameworks and technologies which supported both the rapid prototyping and stable implementation. Because the system incorporates multiple layers, including a React frontend, a Node.js backend, a MongoDB database and two AI components, it was important to utilize a development environment that would allow for consistent testing of all modules. For this reason all major components were implemented on the same workstation to achieve predictable behaviour and to avoid conflicts of different runtime environments.

The frontend was implemented with React.Js (as it was the fastest and a geared technology to build interactive web interfaces, based on the idea of its component-based architecture). React's modular structure made it possible to develop the doctor, nurse and administrator interfaces independently but still share common UI components. During development a modern bundler like Vite or Webpack was used to offer fast local builds, hot module reloading and efficient asset handling. React Router made it possible for client-side navigation and Axios made it possible to communicate stable with the backend API.

The backend was developed using Node js with the express js framework. Node.js was especially appropriate for this purpose because its event-driven architecture is well-suited for handling concurrent API requests, which is critical in an EHR system where multiple users may be accessing or updating data simultaneously. Express gave minimal and flexible way of defining routes, controllers and middleware. Development was supported by Postman which was used in testing API endpoints, validating request and response structures and identifying errors in authentication flows or FHIR resource construction.

For a database we used MongoDB as the main database. MongoDB's document-oriented way of thinking is naturally aligned with FHIR resources, which are also in a format that can be represented in the form of a document which is represented by a type of data structure called a 'JSON' file. Mongoose was used as the Object Document Mapper to define schemas, apply validation rules and manage relationships between patients, appointments, encounters and prescriptions. Since the system was so dependent on consistent identifier creation and schema enforcement, Mongoose's pre-save hooks and middleware functions had an important role when in development.

The AI components needed some more tools. The summarization module relied on the HuggingFace Inference API, which offers pre-trained transformer models that can be used to produce short summaries from clinical text. Since the local development environment did not have the computational power necessary to run large transformer models, the cloud-based interface was necessary in order to get acceptable performance. For the local AI microservice Python 3.10, FastAPI and SentenceTransformers were used. FastAPI allowed us to build this lightweight microservice very quickly and it could be run on CPU with no extra hardware. The detection of drug class and the analysis of prescription risk could be performed by the sentence-transformer model coupled with the rule-based logic and executed on-site and connected to the main backend with the help of the via-http communication.

Version control and collaboration was supported using Git and GitHub to make sure that changes that were made to the system were tracked and that development could safely

proceed without having to lose previously done work. Throughout development, testing tools like browser developer tools, Postman collections and console logging helped to verify data flow and identify problems in real time.

Overall, the tools and technologies chosen enabled the efficient development of the system while maintaining the technical requirements of a modern EHR system. The combination of web development frameworks, database tools and AI libraries created a cohesive environment in which both clinical workflows as well as AI-driven decision support could be implemented and tested effectively.

4.1.7 Data Preparation and Test Scenario Design

The development and evaluation of the MediEase system required carefully prepared test data because real patient information is not possible because of confidentiality and ethical restrictions. For this reason, all patient, practitioner, appointment and encounter records used in the testing were completely synthetic. The synthetic dataset was created to mimic real records of hospital data to ensure that the system could be tested in the same conditions as what would occur in a clinical workflow. Patients were generated using different demographics including different ages, genders and medical histories among them, so the search functions and FHIR-based data structures in the system can be tested extensively. Additional synthetic conditions such as diabetes, asthma and gastritis were assigned to patients in support of the AI prescription validation module testing.

Practitioner accounts were also created to represent the three main system roles - doctors, nurses and administrators. These accounts held appropriate identifiers (e.g. medical licence numbers for doctors and internal staff numbers for nurses). Appointment records were created using different time slots, status and patient-doctor match up to test the backend's capability to manage scheduling logic. Encounter records were created by starting appointments through the frontend and recording sample complaints, vitals, notes and diagnoses. These encounters were prepared to test the behaviour of the summarization component as well as to ensure that the FHIR structure for an encounter was correctly linked to appointments and prescriptions. Some different approaches were taken to populate the system with test data. Initial users and baseline records were inserted using simple seeder scripts so that the system always started from a consistent state. Additional records were created through the frontend in order to test the user interface and form validation. Finally, Postman was used to create bulk test data in the form of repeated API requests which helped to study how the backend behaved under multiple consecutive operations. This blend of manual entry, automatic scripts and API-based creation meant that the data preparation process was both controlled testing conditions and actual operational use.

Once the artificial data set had been prepared, test scenarios were created to simulate a real clinical workflows. Such scenarios comprised full end-to-end scenarios such as registration of a new patient, scheduling an appointment, beginning a consultation, documenting an encounter, issue of a prescription and AI validation of the prescription. Special scenarios were also created to test boundary conditions such as invalid data entry, missing fields, duplicate records and incorrect user roles. This approach ensured that the system was not only evaluated for its expected behaviour but also for how it handled erroneous or incomplete data. By creating test scenarios based on real hospital processes, the system was tested in real-life scenarios that uncovered its capabilities as well as areas that needed improvement.

4.1.8 Experimental Procedures and System Testin

Once the system became filled with synthetic data, a series of experimental procedures were conducted to ensure that each component of the MediEase system worked as intended. Testing was done at a number of different levels, starting with individual modules and progressing to end-to-end workflow tests. The goal was to not only to confirm the correctness of the implementation, but that the system behaved consistently under different components interacting with each other.

The first stage of testing was the validation of individual backend modules. Utility functions, like the ones in charge of generating FHIR resources, of creating sequential identifiers, as well as validating appointment-encounter relationship were tested separately to make sure they returned correct results. Authentication and authorization mechanisms were also validated to ensure that only approved users had access to protected areas of the system. Similar unit-level verification was performed for the local AI microservice to verify that drug classification, risk detection and validation responses were generated correctly for different combinations of medications and patient conditions.

The second stage of the testing was integration testing. This stage ensured communication between modules - like the communication between backend and database, frontend and backend, backend and AI microservice, etc - was working fine. For instance, once an appointment was created, the system was tested to ensure that the doctor could retrieve an appointment and start an encounter and save clinical documentation. Tests were also carried out to ensure that prescriptions generated in the frontend were properly passed to the AI microservice and the warnings being returned were displayed properly.

The final stage included full system and workflow testing. End-to-end scenarios were run multiple times to evaluate stability, performance and usability in a real-world usage situation. These workflow tests included the registration of a new user, the approval of the user by an administrator, logging into the system, creating patients, scheduling appointments, the completion of encounters and creation of clinical summaries by the HuggingFace API. These evaluations helped ensure that the entire platform behaved the same way, even when transitions took place between different parts of the system.

In addition to the functional testing, basic usability testing was carried out, through asking users with a medical background to interact with the prototype. They were encouraged to make comments on layout clarity, navigation structure and form design. This feedback was used to make improvements in the user interface, as well as pointing out where workflows could be simplified. Basic security testing was also done to ensure invalid tokens were rejected, that unauthorized users could not get access to protected routes and sensitive data in requests was handled appropriately. Together, these experimental procedures left little doubt that the system worked reliably and was in line with the research goals.

4.1.9 Implementation Challenges and Workarounds

Throughout development, several challenges were encountered that influenced the design and implementation of MediEase. These challenges arose from technical constraints, interoperability requirements, resource limitations and the complexities of modelling clinical

workflows in software. Each issue required careful analysis and a practical workaround to ensure that the prototype remained functional and met its intended goals.

One major challenge involved the use of HL7 FHIR resources. While FHIR provides a flexible and standardized representation for health data, constructing full FHIR resources manually within controller functions proved time-consuming and prone to mistakes. The nested structure of FHIR models is complex, and early attempts to manually create resources resulted in inconsistency across different modules. This was resolved by introducing dedicated utility functions that automatically constructed FHIR-compliant objects from simpler data inputs. These utilities not only reduced code duplication but also ensured that all resources followed the same structure, improving reliability and maintainability.

Backend design also introduced challenges related to identifier generation. The sequential pattern used for PHN, APID and ENCID required querying the highest existing identifier before creating a new one. While this approach worked well in a single-instance prototype, it was vulnerable to race conditions if multiple requests were processed simultaneously. To reduce this risk, the prototype was run as a single-threaded deployment, and additional checks were added to ensure no duplicate identifiers were produced. In a production environment, a more advanced approach such as atomic counters or UUIDs would be required.

Several difficulties were also encountered when integrating the AI components. Running a large transformer model locally was not possible due to the limited hardware available for development. This made it necessary to rely on the HuggingFace API for summarization. Although effective, this introduced dependency on internet connectivity and required strict de-identification of encounter text to protect patient privacy. The local AI microservice also required fine-tuning to ensure that medication classification and rule-based validation behaved consistently across different drug names and condition combinations. Multiple iterations were necessary to refine the similarity thresholds and rule logic so that the warnings generated were both accurate and clinically meaningful.

Frontend development presented its own challenges. Because the application contained multiple forms and dynamic components, managing state between pages required careful planning. Some UI components needed to re-fetch data to remain consistent, which occasionally resulted in redundant API calls. These issues were reduced by structuring components around clear data-flow rules and using props to pass shared data when possible. Handling PDF report viewing and managing date formats also required adjustments to ensure compatibility with the chosen libraries.

Finally, usability challenges were identified during testing. For example, early versions of the navigation layout caused confusion for users switching between patient lists and encounter views. Based on tester feedback, interface improvements were made by reorganising menu structures and simplifying common workflows.

Chapter 5
Results and Analysis

This chapter presents the results obtained from the design, implementation, and evaluation of the AI-Enhanced Electronic Health Record (EHR) system described in the previous chapters. It brings together the technical outcomes of the prototype, the performance characteristics of the backend and AI components, the feedback from informal usability testing, and the findings from a stakeholder survey on current healthcare record practices and attitudes towards AI-driven solutions in Sri Lanka.

The first part of the chapter reports the results in a structured and factual manner. It describes how the core functional modules behaved under testing, how the system performed in terms of responsiveness, how the AI summarization and prescription validation features operated in practice, and how users experienced the web interfaces during typical workflows. The stakeholder survey results are then presented to show how patients currently interact with hospital record systems, what problems they face with handwritten prescriptions, and how open they are to digital and AI-assisted healthcare.

The second part of the chapter analyses and interprets these findings in relation to the research problem and objectives introduced in Chapter 1, as well as the related work discussed in Chapter 2. It explains how far the implemented system addresses the issues of fragmented records, slow access to information, and unsafe prescribing; what new insights were gained from the survey; what limitations remain in the current prototype; and how these results can inform future work and real-world deployment.

5.1 What is this chapter about

This chapter presents the results obtained from the design, implementation, and evaluation of the AI-Enhanced Electronic Health Record (EHR) system described in the previous chapters. It brings together the technical outcomes of the prototype, the performance characteristics of the backend and AI components, the feedback from informal usability testing, and the findings from a stakeholder survey on current healthcare record practices and attitudes towards AI-driven solutions in Sri Lanka.

The first part of the chapter reports the results in a structured and factual manner. It describes how the core functional modules behaved under testing, how the system performed in terms of responsiveness, how the AI summarization and prescription validation features operated in practice, and how users experienced the web interfaces during typical workflows. The stakeholder survey results are then presented to show how patients currently interact with hospital record systems, what problems they face with handwritten prescriptions, and how open they are to digital and AI-assisted healthcare.

The second part of the chapter analyses and interprets these findings in relation to the research problem and objectives introduced in Chapter 1, as well as the related work discussed in Chapter 2. It explains how far the implemented system addresses the issues of

fragmented records, slow access to information, and unsafe prescribing; what new insights were gained from the survey; what limitations remain in the current prototype; and how these results can inform future work and real-world deployment.

5.2 Results

5.2.1 Functional Results of the System Workflow

Functional testing focused on verifying whether the complete workflow of the system matched the behaviour defined in the requirements and methodology chapters. Starting from registration, the system allowed new patients to be created with their demographic details, contact information and identification numbers. Whenever a new patient was saved, the backend automatically generated a unique Patient Health Number (PHN) using the sequential identifier logic described earlier, and both the PHN and the full FHIR Patient resource were stored successfully in the database. Existing patients could be searched using PHN, NIC or name, and the records were correctly retrieved and displayed in the frontend without inconsistency.

Appointment management behaved according to expectations. Nurses and other authorised staff could create appointments by selecting an existing patient, choosing a doctor, and specifying the date and time. In the doctor interface, only appointments assigned to the logged-in doctor were visible, which confirmed that the role-based filtering worked correctly. When an appointment was used to start a consultation, the system did not delete it; instead, the appointment status was changed to “completed” or “cancelled” depending on the outcome, ensuring that an audit trail of past bookings was always preserved.

Clinical encounters were also handled as planned. A doctor could either start an encounter from an appointment or create a visit directly for walk-in patients. The encounter form captured the chief complaint, vital signs, examination notes, diagnosis and treatment plan in separate fields that matched typical clinical documentation habits. On saving, the backend generated a FHIR Encounter resource, linked it to the corresponding patient and practitioner and stored it in the database, while also exposing the most commonly queried fields through helper attributes. Subsequent requests to view a patient’s history returned the list of encounters in chronological order, confirming that the linking logic worked correctly.

The e-prescription module allowed doctors to create prescriptions within the context of an active encounter. Each prescription contained structured entries for medicine name, dosage, frequency, duration and optional comments. After a prescription was saved, it became part of the patient’s history and could be viewed both from the encounter and from a dedicated prescriptions view. The system also supported embedding key prescription information into a QR code for future retrieval and verification, demonstrating the feasibility of QR-based prescription handling in the Sri Lankan context. A suitable screenshot of the prescription page and QR display can be inserted later as Figure 5.x: E-Prescription Interface and QR Code Output.

Role-based authentication and authorisation were verified through multiple test cases. Users

logged in using their credentials and received a JSON Web Token (JWT), which was then attached to subsequent API requests from the frontend. When a user attempted to access a page that did not match their assigned role, the backend refused the request and returned an appropriate error response. This confirmed that doctors, nurses and administrators were restricted to their intended functionality and that clinical information was not exposed to unauthorised users.

Overall, the functional results show that the core workflow of the system—patient registration, appointment scheduling, encounter documentation, prescription generation and role-based access—operates reliably and in accordance with the design.

5.2.2 Backend, API and Data Handling Results

In addition to basic functionality, tests were conducted to ensure that the backend and database handled data in a FHIR-aligned and performant manner. Each clinical object was stored as a FHIR-style JSON document within a dedicated resource field, while important attributes such as PHN, NIC, appointment identifiers, encounter identifiers and practitioner identifiers were stored at the top level of the document and indexed. During testing, queries for patients by PHN or NIC returned results almost instantly, demonstrating the effectiveness of this hybrid modelling strategy.

API endpoints behaved consistently with the specification. Create operations for patients, appointments, encounters and prescriptions accepted well-formed JSON payloads and returned appropriate success responses, including the generated identifiers and resource structures. When malformed or incomplete data was sent, the server returned clear error messages and did not create partial records, indicating that basic validation mechanisms were functioning correctly. Retrieval endpoints for listing patients, appointments per doctor and encounters per patient all produced the expected results sets.

The connection between the Node.js backend and the local Python AI microservice for prescription validation was also successfully verified. When a doctor created or updated a prescription, the backend formatted the list of medicines and relevant patient conditions and sent this payload to the FastAPI service. The microservice processed the request and returned a JSON response containing risk information, which the backend forwarded to the frontend. No connection failures or blocking delays were observed in the test environment.

5.2.3 Performance Results

Performance was evaluated informally in the local development environment to confirm that the system was responsive enough for interactive clinical use. For common read operations such as loading the list of patients, retrieving an individual patient's profile or listing the upcoming appointments for a doctor, the API responded within a few hundred milliseconds. As these operations were triggered from a React single-page interface, page transitions did not require full reloads, which made navigation feel smooth to users observing the tests.

Create operations, including new patient registration, appointment creation and encounter saving, also returned responses quickly enough to avoid disrupting workflow. No significant slowdowns were observed when several test users interacted with the system sequentially.

While this environment does not simulate the load of a large public hospital, the results suggest that the combination of Node.js, MongoDB and indexed fields is capable of supporting small to medium-scale deployments, provided that appropriate hardware is used.

The local AI microservice for prescription validation added only a minor overhead to prescription creation. The time needed to classify medicines and evaluate risk rules was short enough that warnings appeared almost immediately after the doctor submitted the prescription form. For summarization, the response time depended on the HuggingFace API and network latency. In most tests, summaries were returned within a few seconds, which is acceptable in routine consultations, but this dependence on external services is considered further under limitations.

5.2.4 AI Feature Results

The AI features were implemented at prototype level and evaluated to confirm their technical feasibility and practical usefulness.

For clinical summarization, the backend extracted encounter notes and other relevant clinical text, removed direct identifiers such as names and PHN numbers and sent the de-identified text to the external summarization API. The returned summaries were concise and usually captured the main chronic conditions, recent complaints and key treatments. When displayed in the doctor interface above the detailed history, these summaries allowed users to gain a quick overview of a patient's status without reading through every past note. In some cases, the summaries were somewhat general and omitted subtle details, which suggests that domain-specific training data and fine-tuning would further improve quality. A sample screen showing an encounter with its automatically generated summary can be inserted as Figure 5.x: Example of AI-Generated Clinical Summary.

The prescription validation microservice relied on sentence-embedding based classification of medicine names into drug classes and a rule base that linked drug classes to risky conditions or duplicate entries. During testing with synthetic and example data, the service successfully detected obvious problems, such as the same drug being added twice, a medicine being prescribed despite being recorded as an allergy and specified condition–drug combinations that had been encoded as unsafe in the rules JSON. When these patterns were present, the doctor's interface displayed warning messages before the prescription was finalised, giving the user the opportunity to correct or override the order. This confirms that even a lightweight and local AI component can provide meaningful decision support without requiring a GPU or external services.

5.2.5 Usability Evaluation Results

Informal usability evaluation sessions were carried out with a small group of intended users, including medical students and healthcare staff who were familiar with outpatient workflows. Participants were asked to perform typical tasks such as registering a new patient, scheduling an appointment, starting an encounter, documenting a visit and creating a prescription.

Most users were able to log in and understand their role-specific dashboards with minimal explanation. The layout of the patient list and appointment pages, which displayed key columns such as PHN, name, date and doctor, was considered intuitive, and the search and

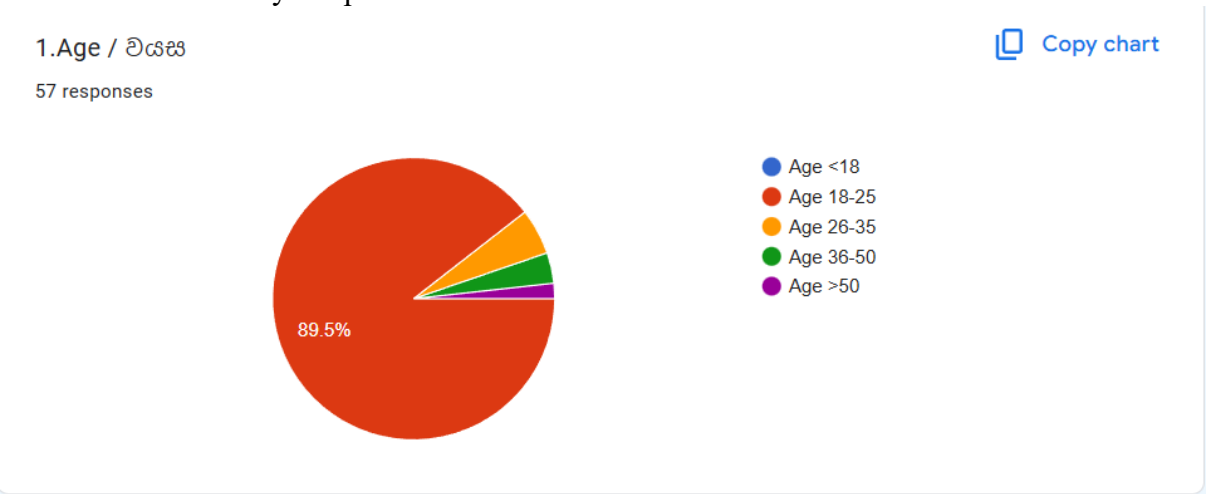
filter boxes allowed users to locate records quickly. On the patient details page, the combination of demographic data, basic health indicators and encounter history provided a clear snapshot of the patient's situation. Users reported that the separation of the encounter form into complaint, vitals, notes and diagnosis matched the way they usually think during a consultation and made it easier to avoid missing important sections.

The e-prescription interface was perceived as clearer than handwritten prescriptions, mainly because medicine entries were structured and the dose and frequency fields were explicitly labelled. However, the evaluation also highlighted some areas for improvement. Some forms did not initially indicate which fields were mandatory, leading to confusion when the system rejected incomplete submissions. Participants suggested that frequently used actions such as "Start Visit" and "Create Prescription" should be more visually prominent. These observations were used to adjust labels, add clearer validation messages and slightly reorganise buttons in the final iteration of the prototype.

5.2.6 Stakeholder Survey Results

To complement the technical tests, a Google Forms survey was conducted with fifty-seven respondents to understand how patients currently interact with health record systems, what problems they experience and how they feel about digital records, QR-based prescriptions and AI tools. Only the most relevant findings are presented here; additional charts can be included in an appendix if required.

The age distribution shows that the majority of respondents, approximately 89.5 per cent, were between 18 and 25 years of age, while only a small fraction were older than 35. This indicates that the sample primarily represents a relatively young and digitally familiar population. A pie chart illustrating this distribution can be added as Figure 5.x: Age Distribution of Survey Respondents.



5.x: Age Distribution of Survey Respondents.

When asked whether they had seen hospitals or clinics using digital systems to store patient records, 38.6 per cent answered "Yes", while 61.4 per cent reported that their records were still handled manually. This suggests that, even among a younger group, full digitalisation of health records is far from universal in the local context. The corresponding chart may be inserted as Figure 5.x: Exposure to Digital Health Record Systems.

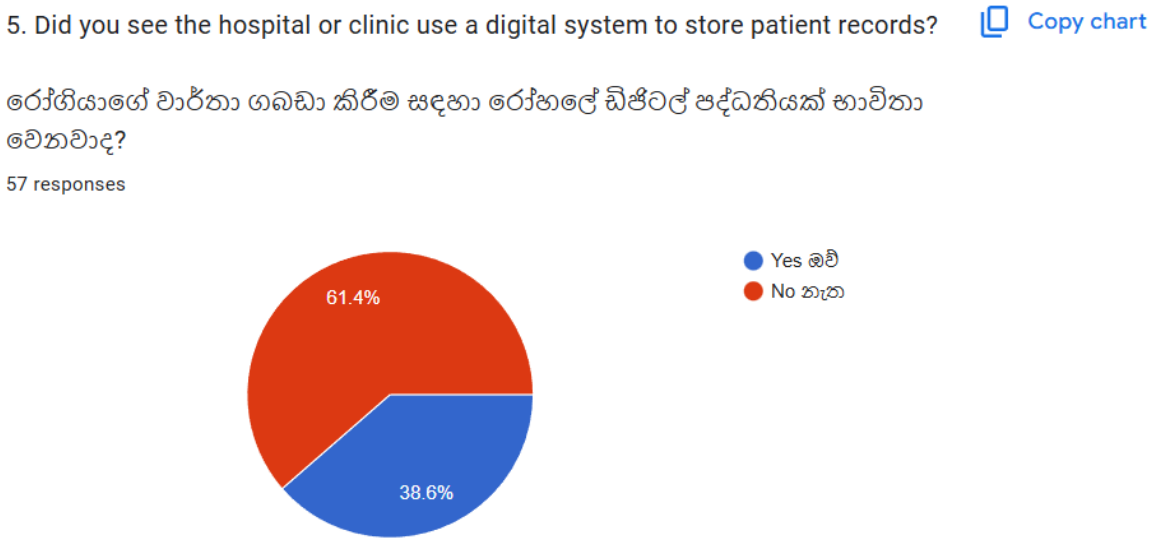


Figure 5.x: Exposure to Digital Health Record Systems.

Respondents identified several problems with the current hospital health record system. Many reported difficulty accessing previous medical records when needed, missing or incomplete records, the burden of repeating the same information at each visit and issues arising from large volumes of paper files. These concerns were reflected in the response distribution, where difficulty accessing medical records was selected by more than half of participants, and missing information and repetition of data were also common. These results directly mirror the fragmentation and inefficiency problems highlighted in the problem statement of this thesis. A bar chart summarizing these responses can be placed as Figure 5.x: Reported Problems with Current Record Systems.

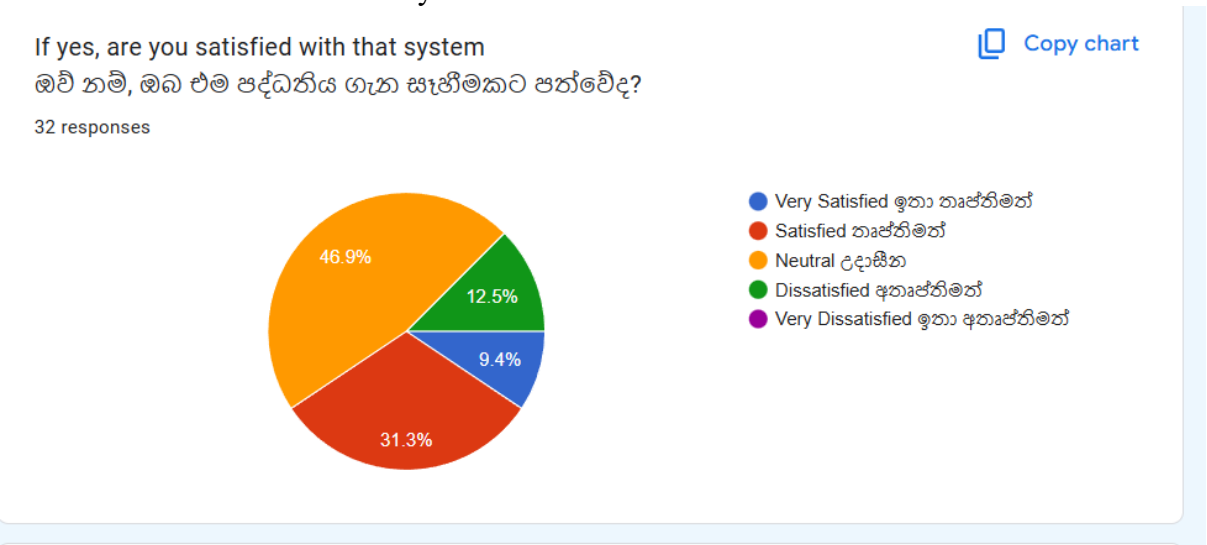
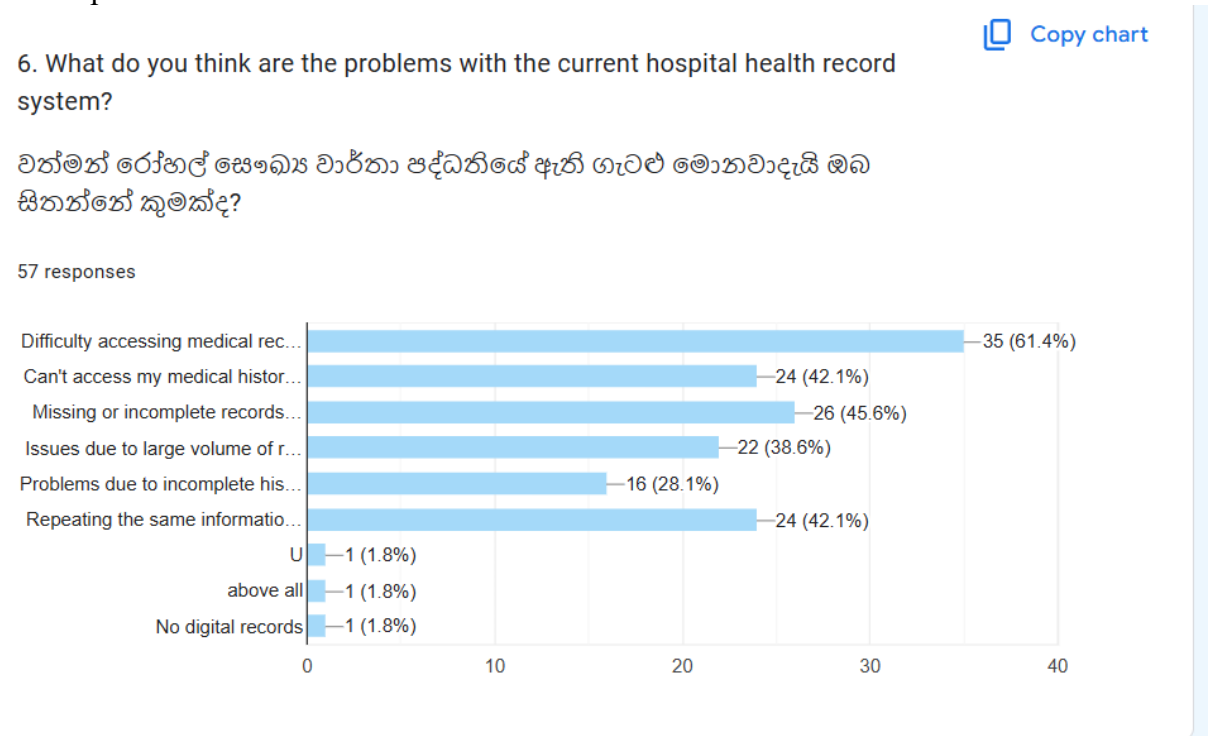


Figure 5.x: Reported Problems with Current Record Systems.

The survey also investigated prescription-related issues. A very high proportion of respondents, over ninety per cent, indicated that illegible handwriting caused problems when receiving prescriptions in writing. Many also mentioned missing information such as dosage

or duration and difficulty understanding medical terminology. This confirms that handwritten prescriptions are a major source of confusion and potential error for patients. The importance of this finding justifies the focus on clear digital prescriptions and QR codes in the proposed system. The chart can be included as Figure 5.x: Problems Experienced with Handwritten Prescriptions.



5.x: Problems Experienced with Handwritten Prescriptions.

When asked about their preferred format, 71.9 per cent reported that they currently receive prescriptions in writing, but 93 per cent stated that they would prefer to receive prescriptions via QR code. This strong preference indicates that patients are open to transitioning from paper-based prescriptions to digital alternatives, provided that the solution is convenient and reliable. A pie chart representing this preference can be shown as Figure 5.x: Preference for QR-Based Prescriptions.

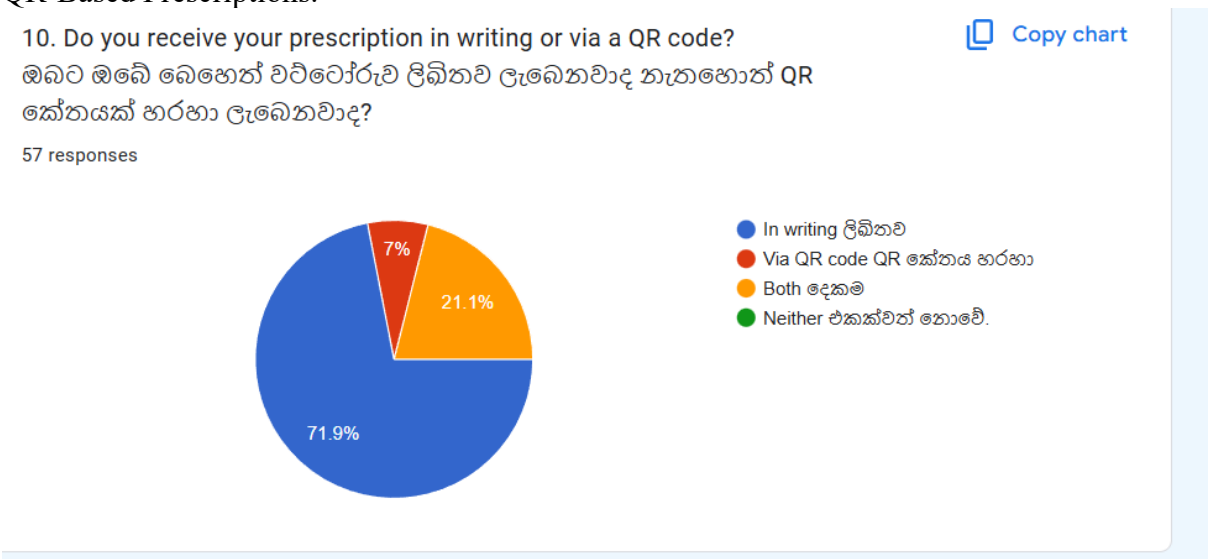


Figure 5.x: Preference for QR-Based Prescriptions.

The survey further explored attitudes towards AI. Around 75.4 per cent of respondents reported having some experience with AI tools in general, mostly through everyday applications. When asked whether it was acceptable for AI tools to assist doctors in making treatment decisions, such as by summarizing medical history, 78.9 per cent agreed that it was acceptable depending on the quality of the AI tool, while only a small minority preferred that doctors make decisions without any AI support. This suggests that the public is generally open to AI-assisted healthcare, although they still expect the doctor to remain the final decision-maker. A chart illustrating this acceptance can be introduced as Figure 5.x: Attitudes towards AI Assistance in Healthcare.

Together, these survey results provide quantitative support for the design choices of the system: they confirm the real-world problems of fragmented records and handwritten prescriptions and indicate that patients are willing to accept digital and AI-enhanced solutions.

5.3 Analysis and Discussion

5.3.1 Achievement of Research Objectives

The functional and survey results together indicate that the main objectives outlined in Chapter 1 have been largely achieved. The system successfully centralises patient data by storing FHIR-style Patient, Appointment and Encounter resources in a single database, reducing fragmentation across multiple paper files and clinics. The ability to retrieve complete histories and display them through the frontend addresses the problem of doctors not having timely access to prior information.

The implementation of an electronic prescription module, combined with optional QR code generation, directly responds to the widespread problems with handwritten prescriptions identified both in the literature and in the survey. Since more than ninety per cent of respondents reported experiencing difficulties with handwritten prescriptions and the vast majority expressed a preference for QR-based prescriptions, the proposed digital solution is both technically feasible and socially acceptable.

The AI summarization component demonstrates that long encounter notes can be transformed into shorter, readable summaries which support doctors during time-pressured consultations. Although the current model operates via a cloud API and uses general-purpose training data, the results show that summarization can highlight important diagnoses and treatments, which aligns with research on transformer-based models in clinical text understanding. Similarly, the local AI prescription validation service confirms that it is possible to perform safety checks without relying entirely on manual review. Even though it currently covers only a limited rule set, it provides proof-of-concept support for the goal of reducing prescription errors.

Finally, the usability findings show that doctors and nurses can carry out their daily tasks—registration, appointment scheduling, encounter documentation and prescribing—within a structured digital workflow that reduces repetition and avoids searching through disorganised paper files. This improvement in workflow efficiency is consistent with the project's broader

objective of improving the quality and safety of care through better information management.

5.3.2 Significance of the Findings

The results are significant for several reasons. Firstly, they demonstrate that an AI-enhanced, FHIR-aligned EHR system can be implemented using open-source technologies and modest hardware, which is crucial in the Sri Lankan context where many hospitals have limited IT infrastructure. The combination of Node.js, MongoDB, React and a lightweight Python microservice shows that advanced functionality can be provided without the need for enterprise-grade servers or expensive vendor systems.

Secondly, the survey confirms that patients are aware of the shortcomings of current systems and are receptive to change. The strong preference for QR-based prescriptions and the high acceptance of AI assistance indicate that the public is ready to adopt new practices as long as they enhance clarity, safety and convenience. This provides a favourable environment for the gradual introduction of digital health solutions such as the one proposed in this thesis.

Thirdly, the integration of AI summarization and local prescription validation into the EHR moves the system beyond simple digital record-keeping. Instead of merely storing information, the system begins to act as a decision-support tool that assists clinicians in managing complex data. This aligns with international trends where EHRs are increasingly expected to provide intelligent support rather than function as passive repositories.

5.3.3 Comparison with Related Work

Chapter 2 highlighted that many EHR implementations in low- and middle-income countries struggle with interoperability, limited AI integration and poor user acceptance. The findings of this study suggest that a carefully designed, FHIR-compliant system with lightweight AI components can overcome some of these barriers. By using FHIR-style resources, the prototype aligns with international data standards and can, in principle, integrate with future national health information exchanges.

Compared with existing local systems that mostly digitise registration and basic encounter data without intelligent features, this system adds summarization and prescription safety checking, demonstrating a practical pathway towards AI-augmented healthcare. The survey results, which reveal persistent problems with access to records and comprehension of prescriptions, reinforce the need to move beyond basic digitisation and towards systems that actively support clinical decisions, as proposed here.

5.3.4 Limitations of the Study

Despite the positive outcomes, several limitations must be acknowledged. The evaluation was conducted using a limited dataset in a controlled environment rather than a live hospital setting. As a result, the performance measurements do not account for the high concurrency, network variability and data volume of a large public institution. Further testing in real deployments is necessary before strong conclusions on scalability can be made.

The AI modules are also at an early stage. The summarization model relies on a general-purpose cloud service and has not been fine-tuned on large collections of local clinical notes, which may limit its clinical accuracy. The prescription validation engine uses a manually defined rule base and a small set of example drug classes; it does not yet integrate with comprehensive drug interaction databases or national formularies. Therefore, it should be treated as a decision-support aid rather than a definitive safety system.

The survey sample is relatively small and skewed towards younger respondents, which may not fully represent the broader population, especially older patients who might have different attitudes towards digital systems and AI. In addition, the usability evaluation involved only a limited number of users and short testing sessions, so longer-term acceptance and training requirements remain to be studied.

5.3.5 Implications and Future Directions

Even with these limitations, the findings have important implications. They show that there is both a technical pathway and a user demand for AI-enhanced EHR systems in Sri Lanka. The current prototype can serve as the basis for pilot deployments in selected clinics or hospital units, where its performance, reliability and impact on workflow can be monitored over time. Data gathered from such pilots would help refine the user interface, expand the AI rule base and identify integration requirements with existing hospital systems.

Future work could focus on several directions. On the technical side, gathering anonymised local clinical text would allow fine-tuning of summarization models and evaluation of their fidelity to clinical meaning. The prescription validation service could be extended with richer pharmacological databases and more sophisticated algorithms capable of detecting multi-drug interactions and dose-related issues. On the systems side, mobile or tablet interfaces could be developed to support bedside use and rural outreach clinics.

From a policy and governance perspective, the FHIR-based design suggests that this system could act as a building block for larger national digital health initiatives. If combined with appropriate security, interoperability and governance frameworks, such systems can contribute to a gradual but meaningful improvement in the quality, safety and efficiency of healthcare delivery in Sri Lanka.

In summary, this chapter has shown that the proposed AI-enhanced EHR prototype functions correctly, is usable in practice, and aligns with the needs and expectations of users, while also highlighting clear directions for future enhancement and deployment.

Chapter 6
Conclusions and Future Works

6.1 Conclusions

This study has aimed to design and develop an AI-Enhanced Electronic Health Record (EHR) system that improves clinical decision-making, reduces workflow inefficiencies, and enhances patient-safety practices within Sri Lankan healthcare settings. The system has been built to address long-standing challenges such as slow access to historical patient information, handwritten documentation, medication errors, and the absence of intelligent digital tools in local hospitals.

Throughout the project, the research has presented a complete and practical solution that integrates FHIR-based data modeling with AI-powered summarization and prescription validation. The system has been implemented as a working prototype using React for the frontend, Node.js for the backend, a MongoDB database, and a Python microservice for local prescription safety checks. The summarization functionality has been achieved using the HuggingFace cloud API with strict de-identification rules to ensure privacy.

The development and evaluation activities have demonstrated that the system has significantly streamlined the documentation process. The AI summarization module has reduced long encounter notes into short, clinically meaningful summaries, improving the speed at which doctors access essential information. Similarly, the prescription validation microservice has provided real-time warnings for high-risk medication decisions, increasing awareness of drug–condition mismatches. These two features have collectively strengthened clinical decision support and addressed key issues identified in the initial problem statement.

The prototype has also supported improvements in workflow efficiency. User testing has shown that patient registration, encounter creation, and data retrieval have become faster and more structured compared to existing semi-digital or paper-based processes in Sri Lankan institutions. The FHIR-compliant data structure has ensured consistency and created a strong foundation for future interoperability with national healthcare systems.

Despite the hardware limitations and the need to depend on cloud-based summarization, the project has successfully delivered a functional, realistic, and technically sound system capable of operating in local hospital environments. The implementation challenges—such as FHIR complexity, identifier generation, AI model tuning and UI workflow issues—have been addressed through iterative refinement. As a result, the final prototype has demonstrated that intelligent EHR systems can be built even with limited resources, and that AI-based decision support can be integrated safely and meaningfully within a clinical workflow.

Overall, this research has contributed a novel prototype that combines FHIR structure, AI text summarization and local prescription validation into one unified system. It has demonstrated the feasibility of enhancing EHR functionality in Sri Lanka using lightweight, modular and

scalable technologies, ultimately showing that AI-driven digital healthcare solutions can be developed even under resource-constrained conditions.

6.2 Future Work

Although the current implementation has met its objectives, several improvements can enhance the system further and support deployment at a national scale. Future work should focus on expanding the system's intelligence, improving scalability and ensuring closer integration with real clinical environments.

A major direction for future enhancement is the development of a fully local summarization model. Running a transformer model locally would eliminate the dependency on internet connectivity and address privacy concerns associated with cloud-based processing. With access to higher-end GPUs or local hospital servers, a fine-tuned medical summarization model could be trained and deployed directly on-site.

Another important improvement involves expanding the prescription validation engine. The current prototype contains a limited set of interaction rules designed for the most clinically relevant scenarios. Future work may include building a more comprehensive medical knowledge base that covers a larger number of drugs, conditions and contraindications. Integrating evidence-based international drug-interaction datasets and supporting Sinhala/Tamil input could make the module significantly more powerful.

Inter-hospital interoperability also represents an important direction. Because the system already follows the FHIR standard, future research can explore how different hospitals can exchange records securely through a national health information exchange (HIE). This would improve continuity of care, reduce repeated tests and allow doctors to access patient history regardless of location.

Performance optimisation and advanced testing are also needed for real-world deployment. Stress testing with hundreds of concurrent users, automated fail-over strategies, audit trails, real-time logging and role-based analytics dashboards can strengthen system reliability. Additional usability studies with clinicians would help refine the interface and ensure that the workflows match daily hospital practices even more closely.

Finally, machine-learning enhancements can be explored to complement summarization and prescription validation. These include predictive risk scoring, automated report extraction, anomaly detection in patient vitals and personalised medication suggestions. Such features would move the system toward becoming a more comprehensive AI-assisted clinical decision-support platform.

In summary, while the current project has delivered a functional and meaningful contribution toward intelligent digital healthcare in Sri Lanka, future work can expand its capabilities, increase accuracy and support large-scale adoption. These advancements will strengthen the system's role in improving patient safety, doctor efficiency and the overall quality of healthcare delivery.

References

1. MedGPT / Clinical Transformer Paper

Original Citation:
Aluísio, S. M., et al. (2024). *Med-GPT: A medical-domain adapted GPT for clinical decision support.* arXiv:2401.13177.

---

2. Large Language Models in Healthcare – Systematic Review

Original Citation:
Jeblick, K., Nisar, S., & Rachinger, M. (2021). *Applications of Deep Learning in Electronic Health Records: A Systematic Review.* Journal of Medical Informatics, 7, e21929.

---

3. NLP for Clinical Decision Support

Original Citation:
Demner-Fushman, D., et al. (2020). *Clinical text mining and decision support: Natural language processing for healthcare.* International Conference on Health Informatics.

---

4. Summarisation in EHR and Clinical Notes (AMIA Paper)

Original Citation:
Enarvi, S., et al. (2019). *Generating Medical Summaries from Electronic Health Record Data.* ACM International Conference on Knowledge Discovery and Data Mining.

---

5. FHIR Standard and Digital Healthcare Interoperability

Original Citation:
Bender, D., & Sartipi, K. (2013). *HL7 FHIR: An agile and RESTful approach to healthcare information exchange.* IEEE CS Conference on Software Engineering.

---

6. AI-Assisted EHR Workflow Optimization

Original Citation:
Liu, X., et al. (2022). *Integrating Machine Learning Models into Clinical Workflows: Opportunities and Challenges.* Journal of Medical Internet Research.